

Adelic reduction of module lattices

Henry Bambury, Seungki Kim, Changmin Lee, Phong Q. Nguyen

Abstract. We give a strict generalization of the LLL algorithm over number fields, based on the reduction theory of $\mathrm{GL}(n)$ over the adèle ring of a number field. Our algorithm is free of heuristics, with rigorous bounds on output quality and complexity. As a consequence, we obtain a hierarchy of reductions from module-(H)SVP to ideal-HSVP, an example of which has runtime and approximation factors subexponential in the field degree. More importantly, we uncover a close connection between structured lattice reduction and a Diophantine approximation over number fields.

1 Introduction

Since structured lattices entered lattice-based cryptography thirty years ago with the NTRU cryptosystem, there has been much temptation to challenge them by exploiting the rich layers of the extra algebraic structure that they possess. Surprisingly, they turned out to be extremely resistant to such attempts, which in no small part led to their remarkable success today, with three out of five NIST PQC standards being based on structured lattices. Though there has been some progress, most notably via S-unit attacks (see e.g., [8,36]); the consensus is that they do not appear to impact the current NIST standards, as NIST Internal Report 8413 [29, p.85] concluded:

...the techniques used in this algorithm rely heavily on the multiplicative structure of ideal lattices, and do not seem to be directly applicable to module lattices of rank 2 or more. Hence these techniques are not known to directly impact the hardness assumptions of any of the round 3 candidates...

One of the most natural ideas for the cryptanalysis of module lattices is an extension of the fundamental LLL algorithm [32] to module lattices. There are other motivations for such an algorithm as well arising from computational number theory in general, *cf.* [38,39]. However, as will be discussed below, all previous attempts at such a generalization are either limited in scope, missing critical details, or reliant on relatively unfamiliar heuristics. The goal of the present paper is to make a contribution in this regard.

1.1 Contributions

Our work is grounded in the philosophy that the LLL algorithm [32] is an algorithmic manifestation of the reduction theory of the moduli space of lattices $\mathrm{GL}(n, \mathbb{Z}) \backslash \mathrm{GL}(n, \mathbb{R})$ — *cf.* [4, Theorem 2.2]. Thus the “correct” module LLL algorithm would be an algorithmic manifestation of the reduction theory of the moduli space of module lattices

(over a number field F), which is known to be

$$\mathrm{GL}(n, F) \backslash \mathrm{GL}(n, \mathbb{A}_F) / \prod_{\nu \nmid \infty} \mathrm{GL}(n, \mathcal{O}_\nu)$$

(see Section 2 below for the notations, also see [7, Sec. 2.3.2]). This space has reduction theory too, in the same sense that $\mathrm{GL}(n, \mathbb{Z}) \backslash \mathrm{GL}(n, \mathbb{R})$ does (see e.g., [20, Ch. 3]). In this paper, we materialize this theory into an algorithm, or, in other words, we give an algorithmic version of this theory.

The outcome, which we refer to as the *adelic LLL* algorithm (Algorithm 8 below), takes as input a $g \in \mathrm{GL}(n, \mathbb{A}_F)$ (instead of $\mathrm{GL}(n, \mathbb{R})$ for LLL), and outputs a reduced $h \in \mathrm{GL}(n, \mathbb{A}_F)$ in the same double coset, that is, there exists $\gamma \in \mathrm{GL}(n, F)$ and $k \in \prod_{\nu \nmid \infty} \mathrm{GL}(n, \mathcal{O}_\nu)$ such that $h = \gamma g k$ and the first row of h is “short”, namely it has low height — an analogue of length in the number field context (*cf.* Section 2.5). It shares with the classical LLL [32] all the important characteristics such as follows.

- Improvement to the given basis g is made by repeatedly size reducing and swapping. In fact, ours is a strict generalization of the original LLL, in that the two algorithms coincide over $\mathbb{Q}$. Just as LLL can be viewed as an efficient algorithmic version of Hermite’s (exponential) inequality on Hermite’s constant γ_n , our adelic LLL can be viewed as an algorithmic version of a Hermite-like inequality on Thunder’s adelic generalization of Hermite’s constant $\gamma_{n,1}(F)$ [41].
- It is completely free of heuristics. In particular, its correctness and bounds on output quality (Propositions 2, 4) and time complexity (Propositions 1, 3 and Theorems 1, 2) can all be rigorously established.
- To elaborate: for LLL, it is well-known that it returns in polynomial time a lattice vector v satisfying

$$\|v\| \leq [(4/3 + \varepsilon)^{1/4}]^{n-1} \cdot |\det L|^{1/n} \text{ and } \|v\| \leq [(4/3 + \varepsilon)^{1/2}]^{n-1} \cdot \lambda_1(L)$$

where L is the lattice and n is its rank. In comparison, adelic LLL returns in polynomial time and polynomially many calls to its subroutines (which one can choose to be polynomial time themselves) an adelic vector $v \in F^n g \setminus \{0\} \subseteq \mathbb{A}_F^n$ such that

$$H(v) \leq Q^{n-1} \cdot H(\det g)^{1/n} \text{ and } H(v) \leq Q^{2(n-1)} \eta(g),$$

where n is the module rank, $\eta(g)$ is a height analogue of $\lambda_1(L)$, and Q is a factor that depends on the number field F . Here, g corresponds to a module lattice, v to a rank 1 submodule of this module lattice, and the height function H provides a way to measure sizes.

- It works over any number field, and for any module lattice of rank ≥ 2 , free or not.
- Its simplicity and explicit parameter values allow for implementation on classical computers and deeper performance analyses.

To the best of our knowledge, no other proposed (structure-preserving) reductions over number fields enjoy all these properties simultaneously — see Section 1.3 and Table 1 below, and Appendix C.

In addition to the robustness of the algorithm, there are a couple of points on the (theoretical) performance of adelic LLL that are worth mentioning:

Reduction from module-(H)SVP to ideal-HSVP Well-known previous results on structure-preserving reductions (e.g., [31,34]) phrased their achievements in terms of reductions from rank $n \geq 2$ module-SVP (resp. Hermite-SVP) to rank ≥ 2 module-SVP (resp. Hermite-SVP). In light of the NIST comment cited above, a reduction to ideal-HSVP would be the most desirable. Yet there existed difficulties involving the reduction of rank 2 modules.

Our work clears this obstacle, thereby obtaining a reduction from approximate module-SVP (resp. HSVP) of any rank to approximate ideal-HSVP, without involving any overly costly oracle that would beat the point of the reduction. The approximation factors and the time complexity are described explicitly in terms of the field invariants and the subroutines used. See Theorem 3 below for details.

Reduction to a Diophantine approximation problem It turns out that the output quality of adelic LLL depends critically on how well one can solve the following Diophantine approximation problem over a number field F : given parameters $0 < \mu < 1$ and $C > 0$, and a vector $(m_\sigma)_{\sigma:F \rightarrow \mathbb{C}} \in F \otimes \mathbb{R}$, find p, q in the ring of integers of F such that

$$|m_\sigma \sigma(q) - \sigma(p)| \leq \mu \text{ and } |\sigma(q)| \leq C^{\frac{1}{[F:\mathbb{Q}]}}$$

for every real or complex embedding σ of F . The smaller we are able to set C (as a function of μ), the better the output of adelic LLL.

In this work, we simply employ a well-known, standard strategy based on (unstructured) lattice reduction (Algorithm 2 below). If one can sufficiently improve on this, however, adelic LLL can outperform the current state-of-art unstructured reductions — e.g., BKZ, slide algorithm — of comparable complexity. In other words, the hardness of the above Diophantine problem is related to whether module lattices are more “vulnerable” than the unstructured ones. Though it turns out that our algorithm performs somewhat worse than BKZ of comparable runtime, it paves an interesting way forward. Halving the currently achieved approximation exponent is sufficient for adelic LLL to match BKZ’s performance; more than half gives it an advantage. See the discussion in Section 4.3 below for details.

We strongly believe that viewing structured lattice reduction from the point of view developed in this paper opens up multiple avenues for further research. We note that as with classical LLL, it is likely that our algorithm performs much better than its theoretical guarantees. Furthermore, any further algorithmic progress on rank-1 module lattices or the adelic subroutines directly improves the approximation factor of adelic LLL.

1.2 Use of adeles

As suggested above, we adopt the language of adeles in this paper. This may make our work somewhat less accessible, but the technical advantages it brings more than make up for this potential entry barrier. For instance, one of the biggest stumbling blocks in previous attempts at generalizing LLL to number fields has been the class group. Hence many works (e.g., [35,18,22,25]) on this subject were limited to the cases in which the

ring of integers is (norm-)Euclidean; also the recent work [10] was primarily devoted to a difficulty of this kind. The adelic perspective allows one to overcome this problem elegantly and effortlessly, without even barely mentioning the class group. This is one of the countless instances in mathematics in which the use of the right language makes the problem drastically more tractable.

Since the adèle ring is defined as an infinite product (see Section 2.2 below), one may wonder how to write an implementable algorithm out of it. When the input comes in the form of a pseudobasis — which is the case for the cryptanalyses of all existing module-based schemes — however, it is possible to rewrite all necessary adelic computations in terms of efficient ideal and vector computations. Thereby we obtain the adelic LLL algorithm (Algorithms 8 and 12 below) that is implementable over classical computers.

Adeles are not exactly new in lattice-based cryptography. They have appeared in the recent work [7]; see also [1, Ch.8] and [12]. A similar construction, sometimes referred to as Arakelov divisors, features in [5] as well. We expect adeles to play more and more important roles in the field, and hope this work may facilitate such development.

1.3 Related work

LLL for modules over number fields has been of interest for a number of reasons since the 1990s. It may be said that a landmark work on this subject is Lee, Pellet–Mary, Stehlé, and Wallet [31]. It was the first, and the only work so far, that attempted to describe a fully structured reduction algorithm over an arbitrary number field, with full details on the algorithm itself as well as its performance analysis. We refer to [31, Sec. 1] for a summary of the literature prior to their work.

The hardest part of [31] is reducing a rank 2 module lattice. Rank ≥ 3 modules are handled in an analogous manner to BKZ, given the rank 2 reduction. Indeed, it seems that the one provided by [31, Section 4] is the only known rank 2 module reduction algorithm prior to the present work. However, it is not an efficient algorithm, because it requires an exact CVP oracle on a lattice of much higher dimension than the input lattice, namely $O(d^{2+\eta})$ (where d is the field degree and $\eta > 0$) versus $2d$. In addition, it relies on several heuristics, and it is a quantum algorithm (unless the input module is free), although it can be dequantized with an additional routine at an extra polynomial-time cost [10, Theorems 6.1-3]. Moreover, some important parameters for the algorithm are not explicitly specified — e.g., c in [31, Lemma 2.5] — presenting a difficulty in its implementation. (Although [21] gives a partial implementation of [31], they completely bypass the rank 2 part by replacing it with BKZ.)

The main differences between the reduction algorithms of [31] and the present work are summarized in Table 1. A more careful comparison, detailing all the aspects in which our work and [31] differ, is given in Appendix C.

Another interesting, oft-cited work on the subject is [34], which generalizes the slide reduction [17] to the context of modules. The most relevant case to us is when the blocksize is 2, which the authors admit is “essentially identical to the reduction in [31]” [34, Sec. 4.1]. They do not substantiate the rank 2 algorithm however. The consideration of the bitsizes is also missing, which is especially important for implementation purposes. The language of filtration introduced in [34] can be read off from the adelic

Table 1. Summary of comparison between [31] and this work. Here, F is the target number field of degree d , and n is the module rank.

	Lee et al. [31]	This work
Quantum/classical	Quantum except on free modules. Dequantization requires an additional algorithm from [10].	Classical
Heuristic	Yes	No
Required oracle	Exact-CVP solver on dimension $d^{2+\eta}$.	Any HSVP solver on dimension $d + \deg E$, for any subfield $E \subseteq F$.
Major contributor to complexity	Preprocessing + poly. calls to CVPP.	Polynomially many calls to the oracle.
Height of short rank-1 module (very roughly)	$2^{(\text{const})nd(\log d)^{O(1)}}$	$2^{(\text{const})nd^2}$, with LLL for HSVP, $\deg E = (\text{const})d$

Iwasawa decomposition (Section 2.6 below), just as a flag of a Euclidean lattice can be read off from the Iwasawa decomposition of the corresponding basis matrix.

[24] (see also the full version [13]) presents a reduction toolkit specialized to free module lattices over cyclotomic fields, especially those of conductor 2^k . They exploit the tower structure of the field (e.g., $\mathbb{Q} \subseteq \mathbb{Q}(\zeta_2) \subseteq \dots \subseteq \mathbb{Q}(\zeta_{2^{k-1}}) \subseteq \mathbb{Q}(\zeta_{2^k})$) and the symplectic structure of certain kinds of lattices (e.g., that of NTRU lattices, as in [16]). These run orthogonal (in the colloquial sense) to the kind of structure that adelic LLL is designed to exploit; perhaps they can complement one another. In any case, the algorithms of [24] are highly heuristic, as the authors admit in [13, Sec. 1.3]. It is also to be noted that they are mainly aimed at being fast, at the expense of the heuristic sub-LLL approximation factor $2^{\tilde{O}(d)^{1/2}}$ on rank 2 module lattices over a cyclotomic field of degree d ; a more precise expression for the approximation factor seems unavailable.

The module BKZ as referred to by in [11], while it does exploit a certain aspect of the module structure, is also tangential to our work, as our main goal is to explore the structured local routine, as in [31], rather than outsourcing it to an unstructured SVP solver.

Outside the cryptographic literature, [39] gives a reduction algorithm of what is essentially rank 2 free modules over the ring of integers $\mathcal{O}_F$ of a totally real field F . Its strongest point is that it returns the representative of some fundamental domain² of the Hilbert modular group $\text{PSL}(2, \mathcal{O}_F)$, thanks to its relatively simple reduction

¹ $\tilde{O}(d)$ means $O(d(\log d)^c)$ for some constant $c > 0$.

² Thus the representative may not consist of short vectors of the module.

theory. Its performance analysis is incomplete, however, except that [39] mentions the complexity is exponential in the input bitsize.

Yet another noteworthy work is [37], which presents an LLL-like reduction algorithm for $\mathbf{G}(\mathbb{Z}) \backslash \mathbf{G}(\mathbb{R})$, where $\mathbf{G}$ is any split semisimple group over $\mathbb{Z}$. From the reduction theory perspective, [37] comes very close in spirit to our work. However, it does not account for the general number field case of $\mathbf{G}(\mathcal{O}_F) \backslash \mathbf{G}(F \otimes \mathbb{R})$ — one may suggest the Weil restriction, but it breaks both splitness and semisimplicity in general. The ideas presented in this paper are precisely the necessary ingredients to make this happen.

2 Preliminaries

The goal of this section is to quickly list the number theoretic notions used in this paper. We seek to deliver the minimum amount of practical knowledge necessary to understand the rest of the paper. The reader is referred to standard texts in algebraic number theory, such as [43], for full details, or to works in cryptology such as [7,12].

Throughout we fix a number field $F/\mathbb{Q}$ of degree d , and write $\mathcal{O}_F$ for its ring of integers.

2.1 Basic number field facts

A *place* ν of F is an equivalence class of absolute values, or metrics, defined on F , where two absolute values $|\cdot|_1, |\cdot|_2$ are said to be equivalent if $|x|_1 < 1 \Leftrightarrow |x|_2 < 1$ for all $x \in F$. The completion of F with respect to ν is denoted by F_ν . In particular, F is a subset of F_ν , and there exists the natural embedding $F \hookrightarrow F_\nu$ by inclusion. For $a \in F$, we often denote by $a_\nu \in F_\nu$ the image of a via this map. We write $\mathcal{P} = \mathcal{P}(\mathcal{F})$ for the set of all places of F . There are two categories of places, *finite* and *infinite*.

The finite places, denoted $\nu \nmid \infty$, correspond one-to-one to the nonzero prime ideals of $\mathcal{O}_F$. For this reason, we will use the notations for places and primes interchangeably, or even refer to places as primes and vice versa, as is customary in the literature. For each $\nu \nmid \infty$, there exists the *ring of ν -adic integers* $\mathcal{O}_\nu \subseteq F_\nu$, the set of all elements of F_ν with absolute value ≤ 1 . $\mathcal{O}_\nu$ is a principal ideal domain whose unique prime ideal is $\nu\mathcal{O}_\nu$; its field of fractions is precisely F_ν . We fix once and for all a generator π_ν of $\nu\mathcal{O}_\nu$, called a *uniformizer* of F_ν . Every element a_ν of F_ν has a unique presentation $a_\nu = \gamma\pi_\nu^\alpha$, where $\gamma \in \mathcal{O}_\nu^*$ and $\alpha \in \mathbb{Z}$; moreover, $a_\nu \in \mathcal{O}_\nu$ if and only if $\alpha \geq 0$. We define $\text{ord}_\nu a_\nu := \alpha$, and $N(\nu) = |\mathcal{O}_\nu : \nu\mathcal{O}_\nu|$. The absolute value corresponding to ν is then $|a_\nu|_\nu = N(\nu)^{-\text{ord}_\nu a_\nu}$. We extend $N()$ to fractional ideals, to denote their norm.

Example If $F = \mathbb{Q}$ and $\nu = p$ is a rational prime, then $F_\nu = \mathbb{Q}_p$ is the field of p -adic numbers, and $\mathcal{O}_\nu = \mathbb{Z}_p$ is the ring of p -adic integers. For $a \in \mathbb{Q}$ or $\mathbb{Q}_p$, $\text{ord}_p a$ is simply the power of p in the prime factorization of a .

An infinite place (or prime), denoted $\sigma \mid \infty$, corresponds to either a real field embedding $F \rightarrow \mathbb{R}$, or a conjugate pair of complex embeddings $F \rightarrow \mathbb{C}$. Let r_1, r_2 be the number of real and complex embeddings, respectively. Following the common convention, we let $\sigma_1, \dots, \sigma_{r_1}$ denote the real embeddings, and $\sigma_{r_1+j} = \bar{\sigma}_{r_1+r_2+j}$ for

$1 \leq j \leq r_2$ denote the pairs of complex embeddings. To clarify, F is a field of degree $d = r_1 + 2r_2$, and has $r = r_1 + r_2$ infinite places. If σ is real (complex), $F_\sigma \cong \mathbb{R}$ ($\cong \mathbb{C}$). The corresponding absolute value, in both real and complex cases, is $|a|_\sigma = |\sigma(a)|$.

Example If $F = \mathbb{Q}$, there is only one infinite place, corresponding to $\mathbb{Q} \hookrightarrow \mathbb{R}$ by $x \mapsto x$. If F is a real quadratic field, it has two real places, corresponding to $x \mapsto x$ and $x \mapsto \bar{x}$, where $\bar{x}$ is the Galois conjugate of x . If F is an imaginary quadratic field, these two embeddings form a conjugate pair of a single complex place of F .

For any $\mathbb{Z}$ -basis $\{b_1, \dots, b_d\}$ of $\mathcal{O}_F$, the *discriminant* of F is the quantity

$$\Delta_F = (\det(\sigma_i(b_j))_{1 \leq i, j \leq d})^2.$$

This definition is independent of the choice of the basis. There exists also the *logarithm map* $\text{Log} : F^* \rightarrow \mathbb{R}^r$ defined by

$$\text{Log}(x) = (\log |\sigma_1(x)|, \dots, \log |\sigma_{r_1}(x)|, 2 \log |\sigma_{r_1+1}(x)|, \dots, 2 \log |\sigma_{r_1+r_2}(x)|).$$

Log takes $\mathcal{O}_F^*$ to a lattice in $\mathbb{R}^r$ of rank $r - 1$, called the *unit lattice*; it is isomorphic to $\mathcal{O}_F^* / \mu_F$, where μ_F is the group of the roots of unity in $\mathcal{O}_F$. Its covolume, with respect to the standard metric on $\mathbb{R}^r$, is $R_F \sqrt{r}$, where R_F is called the *regulator* of F .

2.2 Adeles

The *adele ring* $\mathbb{A}_F$ is defined to be the subset of the product $\prod_{\nu \in \mathcal{P}} F_\nu$ consisting of elements $(a_\nu)_{\nu \in \mathcal{P}}$ such that all $a_\nu \in \mathcal{O}_\nu$ for all but finitely many $\nu \nmid \infty$. Here $(a_\nu)_{\nu \in \mathcal{P}}$ is referred to as an *adele*. The natural embeddings $F \hookrightarrow F_\nu$ induce another natural embedding $F \hookrightarrow \mathbb{A}_F$ by $a \mapsto (a_\nu)$. Henceforth we identify F with the image of this map.

$\mathbb{A}_F$ is equipped with the topology with a basis consisting of sets of the form $\prod_\nu O_\nu$, where O_ν is an open subset of F_ν , and $O_\nu = \mathcal{O}_\nu$ for all but finitely many finite ν .

The set of *finite adeles* $\mathbb{A}_f$ is defined analogously, except that it is a subset of the product $\prod_{\nu \nmid \infty} F_\nu$ over the finite places of F . The set of *infinite adeles* $\mathbb{A}_\infty$ is simply the (finite) product $\prod_{\nu \mid \infty} F_\nu$.

For $x \in \mathbb{A}_F$, we denote by x_f and x_∞ the images of x by the natural projections $\mathbb{A}_F \rightarrow \mathbb{A}_f$ and $\mathbb{A}_F \rightarrow \mathbb{A}_\infty$, respectively. We extend this notation naturally to other adelic constructs like $\mathbb{A}_F^n$ and $\text{GL}(n, \mathbb{A}_F)$.

The *product formula* states that, for $a \in F^*$,

$$\prod_{\nu \nmid \infty} |a_\nu|_\nu \cdot \prod_{\sigma \mid \infty} |a_\sigma|^{e_\sigma} = 1,$$

where $e_\sigma = 1$ or 2 according to whether σ is real or complex.

2.3 Metric on $\mathbb{A}_\infty$

Recall the canonical embedding ρ of F into $\mathbb{R}^{r_1} \times \mathbb{C}^{2r_2}$, defined by

$$\rho(x) = (\sigma_1(x), \dots, \sigma_d(x)).$$

The image $\rho(F)$ spans a vector space of dimension d over $\mathbb{R}$, which we can identify with three sets at the same time: the subspace (isomorphic to $\mathbb{R}^{r_1} \times \mathbb{C}^{r_2}$) of $\mathbb{R}^{r_1} \times \mathbb{C}^{2r_2}$ consisting of all elements whose i -th and $(i + r_2)$ -th complex entries are conjugates for all i , the tensor product $F \otimes_{\mathbb{Q}} \mathbb{R}$, and the set of infinite adeles $\mathbb{A}_\infty$. For $x \in F$ or $\mathbb{A}_F$, we sometimes write x_∞ for the corresponding element of $\mathbb{A}_\infty$.

We endow an inner product on $\mathbb{A}_\infty \cong F \otimes_{\mathbb{Q}} \mathbb{R}$ by restricting to it the standard inner product on $\mathbb{R}^{r_1} \times \mathbb{C}^{2r_2}$, so that

$$\langle \rho(x), \rho(y) \rangle = \sum_{i=1}^d \sigma_i(x) \bar{\sigma}_i(y),$$

and

$$\|\rho(x)\|^2 = \langle \rho(x), \rho(x) \rangle = \sum_{i=1}^d |\sigma_i(x)|^2.$$

We extend the embedding ρ , the above inner product and the norm to the multi-dimensional counterparts in the natural way, and denote them by the same notations.

2.4 How an adele gives a lattice

For $g \in \mathrm{GL}(n, \mathbb{A}_F)$, the *module lattice* associated to g is given by (see e.g. [3])

$$L(g) = \{(xg)_\infty \in \mathbb{A}_\infty^n : x \in F^n, (xg)_\nu \in \mathcal{O}_\nu^n \text{ for all finite } \nu\},$$

whose first minimum is $\lambda_1(L(g)) = \min_{u \in L(g), u \neq 0} \|u\|$. The following lemma is immediate from this definition.

Lemma 1. *For any $h \in \prod_{\nu \nmid \infty} \mathrm{GL}(n, \mathcal{O}_\nu)$ and $k \in \mathrm{GL}(n, F)$, $L(g) = L(kgh)$.*

Remark 1. In fact, it holds that $L(g) = L(g')$ if and only if g, g' represent the same coset in

$$\mathrm{GL}(n, F) \backslash \mathrm{GL}(n, \mathbb{A}_F) / \prod_{\nu \nmid \infty} \mathrm{GL}(n, \mathcal{O}_\nu).$$

See e.g., [7, Sec. 2.3.2] (where they factor on the right by a little more to normalize the determinant and rotational factor).

In the classical language, suppose we are given a pseudobasis of a module lattice of the form $M = \mathbf{b}_1 \mathbf{v}_1 \oplus \dots \oplus \mathbf{b}_n \mathbf{v}_n$ where $\mathbf{v}_i \in (F \otimes \mathbb{R})^n$, say. In the adelic language, M is precisely $L(g)$, where

$$g_\infty = \begin{pmatrix} - & \mathbf{v}_1 & - \\ & \vdots & \\ - & \mathbf{v}_n & - \end{pmatrix}, g_\nu = \begin{pmatrix} \pi_\nu^{-\mathrm{ord}_\nu \mathbf{b}_1} & & \\ & \ddots & \\ & & \pi_\nu^{-\mathrm{ord}_\nu \mathbf{b}_n} \end{pmatrix}$$

for each finite ν .

2.5 Height

Define the (local) *height* at ν of $v_\nu = (a_1, \dots, a_n) \in F_\nu^n$ by

$$H_\nu(v_\nu) = \begin{cases} \max_i N(\nu)^{-\text{ord}_\nu a_i} & \text{if } \nu \text{ is finite} \\ \sqrt{a_1^2 + \dots + a_n^2} & \text{if } \nu \text{ is real} \\ |a_1|^2 + \dots + |a_n|^2 & \text{if } \nu \text{ is complex.} \end{cases}$$

For $v \in \mathbb{A}_F^n$, its (global) *height* is defined to be

$$H(v) = \prod_{\nu \in \mathcal{P}} H_\nu(v_\nu).$$

The product formula implies that $H(v) = H(rv)$ for any $r \in F^*$.

Sometimes it is convenient to separate the finite and infinite parts of the height. We denote them by $H_{\mathbf{f}} = \prod_{\nu \nmid \infty} H_\nu$ and $H_\infty = \prod_{\nu \mid \infty} H_\nu$, respectively.

Lemma 2. *For $g \in \text{GL}(n, \mathbb{A}_F)$, $\det L(g) = |\Delta_F|^{\frac{n}{2}} H(\det g)$. Here $\det g = \prod_\nu \det g_\nu$.*

Proof. This follows from e.g. [30, Lemma 2.3]. $\square$

We introduce the minimal height (which corresponds to the adelic first minimum introduced in [3]): for $g \in \text{GL}(n, \mathbb{A}_F)$, let

$$\eta(g) = \min_{v \in F^n g \setminus \{0\}} H(v).$$

It is possible to view η from the perspective of pseudobases:

Lemma 3. *Let M be a module lattice given by a pseudo basis $M = \mathbf{b}_1 \mathbf{v}_1 \oplus \dots \oplus \mathbf{b}_n \mathbf{v}_n$ where the $\mathbf{v}_i \in (F \otimes \mathbb{R})^n$'s are linearly independent, and the $\mathbf{b}_i$'s are fractional ideals. Let $g \in \text{GL}(n, \mathbb{A}_F)$ such that $M = L(g)$. Then:*

$$\eta(g) = \min_{\substack{\text{nonzero } \mathbf{v} \in (F \otimes \mathbb{R})^n \\ \text{fractional ideal } \mathbf{b}, \mathbf{b}\mathbf{v} \subseteq M}} N(\mathbf{b}) H_\infty(\mathbf{v}),$$

which is $\leq \min_{1 \leq i \leq n} N(\mathbf{b}_i) H_\infty(\mathbf{v}_i)$. Furthermore:

$$d^{-d/2} |\Delta_F|^{-1/2} \lambda_1(M)^d \leq \eta(g) \leq d^{-d/2} \lambda_1(M)^d \leq n^{d/2} (\det M)^{1/n}$$

Proof. [31, Lemma 2.2] proved inequalities relating $\lambda_1(M)$ and a quantity called $\lambda_1^N(M)$, which is slightly different from $\eta(g)$, but the same arguments show our inequalities. $\square$

The last inequalities of Lemma 3 show that finding a short vector in M (with respect to either λ_1 or $\det$) can be reduced to finding a short vector in the rank-1 module lattice $\mathbf{b}_1 \mathbf{v}_1$ (with respect to its $\det$), provided that $N(\mathbf{b}_1) H_\infty(\mathbf{v}_1)$ is close to $\eta(g)$.

2.6 Iwasawa decomposition in $\mathrm{GL}(n, \mathbb{A}_F)$

For any place ν , $\mathrm{GL}(n, F_\nu)$ admits an *Iwasawa decomposition* $g_\nu = u_\nu a_\nu k_\nu$, where u_ν is a lower-triangular unipotent matrix, a_ν is diagonal and with positive real entries when $\nu \mid \infty$, and $k_\nu \in K_\nu$, where $K_\nu = \mathrm{GL}(n, \mathcal{O}_\nu)$, $\mathrm{O}(n, \mathbb{R})$ or $\mathrm{U}(n, \mathbb{C})$ according to whether ν is finite, real, or complex. In the real or complex case, computing this decomposition comes down to the Gram-Schmidt orthogonalization. Note that, over the infinite places, this coincides with the Gram-Schmidt orthogonalization on $(F \otimes \mathbb{R})^n$ commonly seen in the cryptology literature (e.g. [31, Sec. 2.3]). When $\nu \nmid \infty$, the decomposition can be made unique by dictating that the diagonal entries $a_{i,\nu}$ of a_ν are powers of π_ν , and that the (i, j) -entry of u_ν is to vanish under the natural projection $F_\nu \rightarrow (a_{i,\nu}/a_{j,\nu})\mathcal{O}_\nu$ — cf. [20, Claim 2.2.1].

In case $n = 2$ for example, collecting the local decompositions together, one obtains an Iwasawa decomposition of $g \in \mathrm{GL}(2, \mathbb{A}_F)$, where

$$g = uak, \quad u = \begin{pmatrix} 1 & \\ m_{21} & 1 \end{pmatrix}, \quad a = \begin{pmatrix} a_1 & \\ & a_2 \end{pmatrix}, \quad k \in K := \prod_\nu K_\nu. \quad (1)$$

Here m_{12} is an adele, and a_1, a_2 are *ideles*, i.e. $(a_i)_\nu \in \mathcal{O}_\nu^*$ for all but finitely many $\nu \nmid \infty$. Below, whenever we discuss an element $g \in \mathrm{GL}(n, \mathbb{A}_F)$, the choice of the Iwasawa decomposition $g = uak$ will be given from the beginning, which is to be updated as it undergoes changes by our algorithm. Additionally, we write v_i for the i -th row vector of g , and $\alpha_i = a_{i+1}/a_i$. In the $n = 2$ case, we drop some of the subscripts, when there exists no room for ambiguity.

We state and prove a lemma generalizing the fact that any nonzero vector of a lattice spanned by the rows of $g \in \mathrm{GL}(n, \mathbb{R})$ has length at least the minimal Gram-Schmidt coefficient of g .

Lemma 4. *Let $g \in \mathrm{GL}(n, \mathbb{A}_F)$ have an Iwasawa decomposition $g = uak$, and denote $a = \mathrm{diag}(a_1, \dots, a_n)$. Then*

$$\eta(g) \geq \min_{1 \leq i \leq n} H(a_i).$$

Proof. Choose a vector $w = (w_1, \dots, w_n) \in F^n$, and let j be the largest index such that $w_j \neq 0$. First, notice that for each place ν , right multiplication by K_ν preserves the local height at ν . Therefore $H(wuak) = H(wua)$. Since the j -th entry of wua is $w_j a_j$, we have

$$H(wua) \geq H(w_j a_j) = H(a_j),$$

where the last equality is by the product formula. $\square$

2.7 Complexity

We need to clarify the meanings of certain expressions to discuss the complexity of our algorithms below. Assume the knowledge of a basis $\mathcal{B}$ of $\mathcal{O}_F$. An element $x \in F$ can be written as a $\mathbb{Q}$ -linear combination of the elements of $\mathcal{B}$; by the *bitsize* of x ,

we mean the number of bits needed to record this linear combination. Similarly, a fractional ideal of F can be represented as its $\mathbb{Z}$ -basis, which consists of d elements of F ; accordingly, its bitsize is the number of bits needed to record those d elements. The computer representation of adeles will be discussed later, see section 3.3.

By $\text{poly}(x, y, z, \dots)$ we mean a polynomial in variables $x, y, z, \dots$. In this context, $\text{poly}(\text{input})$ means a polynomial in the total bitsize of all necessary input data for the algorithm under discussion.

3 Adelic LLL in rank 2

3.1 Reduction notion

Fix the parameters $0 < \mu, \delta < 1$ such that $\delta^{\frac{2}{d}} - \mu^2 > 0$. We need other parameters $A, B, C > 0$; their choice affects the quality-complexity tradeoff, and will be discussed more carefully below. Recall that we use the convention that m and α are implicitly defined via the corresponding Iwasawa decomposition, such as (1).

Definition 1 (Rank 2 reduced basis). *We say that $g \in \text{GL}(2, \mathbb{A}_F)$ is (LLL-)reduced with respect to the above parameters if it has an Iwasawa decomposition $g = uak$ for which the following points hold*

(i) (size reduced) *There exists $\tau_\sigma > 0$ for each of the d embeddings $\sigma : F \hookrightarrow \mathbb{C}$ such that*

$$\tau_\sigma \leq C^{\frac{1}{d}}, \quad \tau_\sigma |m_\sigma| \leq \mu \quad \text{and} \quad H_f(m, 1) \leq \prod_{\sigma: F \hookrightarrow \mathbb{C}} \tau_\sigma.$$

(ii) (Lovász condition) $\delta \leq H((m, \alpha))$, or equivalently $\delta H((a_1, 0)) \leq H((ma_1, a_2))$.

(iii) (unit reduced) *The entries of*

$$\text{Log}(\alpha_\infty) - \frac{1}{d} \log H_\infty(\alpha_\infty) \cdot \underbrace{(1, \dots, 1)}_{\text{real}} \cdot \underbrace{(2, \dots, 2)}_{\text{complex}}$$

are bounded by A .

(iv) (class reduced) α is integral (that is, $\alpha_\nu \in \mathcal{O}_\nu$ for all finite ν) and $B^{-1} \leq H_f(\alpha)$.

This definition (as well as its rank n counterpart Definition 2) is inspired as much by the reduction theory of $\text{GL}(n, \mathbb{A}_F)$ (see e.g. [20, Sec. 3.3]) as by the original LLL algorithm [32]. Indeed, just as the set of the classical LLL bases approximates the fundamental domain of $\text{SL}(n, \mathbb{Z}) \backslash \text{SL}(n, \mathbb{R})$, the set of reduced bases as defined in this paper approximates the fundamental domain of $\text{GL}(n, F) \backslash \text{GL}(n, \mathbb{A}_F) / \prod_{\nu \nmid \infty} \text{GL}(n, \mathcal{O}_\nu)$. The corresponding conditions for the classical LLL algorithm [32] are consistent with the above in case $F = \mathbb{Q}$ and $A = 0, B = 1, C = 1$, other than the minor difference that δ here is δ^2 in some references.

For (ii), we could have also used the Siegel condition as in [31], but just as in the classical case, the Lovász condition is more likely to yield superior output quality in practice, since it induces a strictly stronger reduction notion.

It is possible to define the reduction notion with respect to a unique choice of Iwasawa decomposition, such as the one discussed in Section 2.6 above. Though this

would lead to a theoretically more satisfying version of adelic LLL, it is not at all necessary for our algorithm to work, and may likely add additional complications to the procedure.

3.2 Overview of the algorithm

We continue with the notations set up above. In addition, we denote the necessary subroutines as follows:

- **Lovasz**(g, δ) returns **true** if g satisfies the Lovász condition, and **false** otherwise.
- **class-reduce**(g, B) ensures that α is integral and $B^{-1} \leq H_{\mathbf{f}}(\alpha)$.
- **unit-reduce**(g, A) ensures that α is unit reduced with respect to A , while preserving class reducedness.
- **size-reduce**(g, μ, C) ensures that g is size reduced with respect to the given parameters, while preserving class and unit reducedness.

When the parameters are clear from the context, we omit those and simply write **size-reduce**(g), and so on.

The last three operations amount to multiplying g from the left by an element of $\text{GL}(2, F)$: for **class-reduce**, by a diagonal matrix; for **unit-reduce**, by a diagonal matrix with unit entries; for **size-reduce**, by a lower unipotent matrix. This helps one see the necessary preservation properties. In the following section, we will substantiate these functions in case the input is given by a pseudobasis.

Algorithm 1 provides a high-level outline of the adelic LLL. At this point, we already have sufficient detail for us to prove its termination and output quality bounds.

Algorithm 1 Adelic LLL algorithm rank 2 (conceptual version)

Require: Parameters δ, μ, A, B, C and $g = uak \in \text{GL}(2, \mathbb{A}_F)$.

Ensure: LLL-reduced $g \in \text{GL}(2, \mathbb{A}_F)$.

```

1: while true do
2:   class-reduce( $g$ )
3:   unit-reduce( $g$ )
4:   size-reduce( $g$ )
5:   if(Lovasz( $g$ )) break loop
6:   else swap the rows of  $g$ 
7: return  $g$ 
```

Proposition 1 (Rank 2 termination). *Algorithm 1 terminates. In fact, if for some $\kappa > 0$, $\eta(g) \geq \kappa$, then it takes at most*

$$\frac{\log H(v_1) - \log \kappa}{-\log \delta}$$

swaps to terminate. (Recall that we are denoting by v_1 the first row vector of g .)

The above statement and its proof, as well as its generalization Lemma 3 in the next section, exactly parallel their classical counterparts. Just as with the classical LLL, if we assume the input g is given by an integral matrix, we can take $\kappa = 1$. In general, if g corresponds to a pseudobasis $\mathbf{b}_1\mathbf{v}_1 + \mathbf{b}_2\mathbf{v}_2$ where $\mathbf{v}_1, \mathbf{v}_2 \in \rho(\mathcal{O}_F^2)$, then we can take $\kappa = N(\mathbf{b}_1 + \mathbf{b}_2)$, as shown in Lemma 5 below.

Proof. From the descriptions provided above, it is clear that only Line 6, the row swapping, of Algorithm 1 has the possibility of changing $E(g)$.

When a swap occurs, $H(v_1) = H((a_1, 0))$ is replaced by $H(v_2) = H((ma_1, a_2))$. Since the Lovász condition fails, we have $\delta H(v_1) > H(v_2)$, and thus $H(v_1)$ decreases at least by a factor of δ . Since $H(v_1)$ cannot be lower than κ , this completes the proof. $\square$

Lemma 5. *Suppose $g \in \text{GL}(2, \mathbb{A}_F)$ corresponds to the pseudobasis $\mathbf{b}_1\mathbf{v}_1 + \mathbf{b}_2\mathbf{v}_2$ where $\mathbf{v}_1, \mathbf{v}_2 \in \rho(\mathcal{O}_F^2)$, i.e.,*

$$g_\infty = \begin{pmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \end{pmatrix}, g_\nu = \begin{pmatrix} \pi_\nu^{-\text{ord}_\nu \mathbf{b}_1} & \\ & \pi_\nu^{-\text{ord}_\nu \mathbf{b}_2} \end{pmatrix}$$

for each $\nu \nmid \infty$. Then $\eta(g) \geq N(\mathbf{b}_1 + \mathbf{b}_2)$.

Proof. Let us write β_i for the finite adele such that $(\beta_i)_\nu = \pi_\nu^{\text{ord}_\nu \mathbf{b}_i}$, so that

$$g_{\mathbf{f}} = \begin{pmatrix} \beta_1^{-1} & \\ & \beta_2^{-1} \end{pmatrix}.$$

Also let $\beta \in \mathbb{A}_{\mathbf{f}}$ be such that $\beta_\nu = \pi_\nu^{\min(\text{ord}_\nu \mathbf{b}_1, \text{ord}_\nu \mathbf{b}_2)}$.

Take any $x_1, x_2 \in F$, not both zero. We have

$$H((x_1, x_2)g) = H_{\mathbf{f}}((x_1\beta_1^{-1}, x_2\beta_2^{-1}))H_\infty(x_1\mathbf{v}_1 + x_2\mathbf{v}_2).$$

First notice

$$H_{\mathbf{f}}((x_1\beta_1^{-1}, x_2\beta_2^{-1})) \geq H_{\mathbf{f}}(x_1\beta^{-1}, x_2\beta^{-1}) \geq H_{\mathbf{f}}((x_1, x_2))H_{\mathbf{f}}(\beta^{-1}).$$

As for $H_\infty(x_1\mathbf{v}_1 + x_2\mathbf{v}_2)$, by considering a single nonzero coordinate of $x_1\mathbf{v}_1 + x_2\mathbf{v}_2$, it is bounded from below by $H_\infty(y)$, where y is some element of the ideal generated by x_1, x_2 . By the product formula,

$$1 = H_{\mathbf{f}}(y)H_\infty(y) \leq H_{\mathbf{f}}((x_1, x_2))H_\infty(y).$$

Combining the last two bounds, we have $H((x_1, x_2)g) \geq H_{\mathbf{f}}(\beta^{-1})$, which completes the proof. $\square$

The following proposition shows that the first row vector of reduced g is guaranteed to be short (in the sense of height) both absolutely (4) and relatively (5), as with the classical LLL. It does so by establishing an adelic analogue (3) of the well-known fact that in an LLL-reduced basis, the norms of the Gram-Schmidt vectors decrease at most geometrically:

Proposition 2 (Rank 2 quality). *Let A, B, C denote the parameters from Algorithm 1 and define*

$$Q = (\delta^{2/d} - \mu^2)^{-d/4} (CB)^{1/2} e^{Ad/2}. \quad (2)$$

If g is reduced, then

$$\frac{H(a_1)}{H(a_2)} \leq Q^2, \quad (3)$$

$$H(v_1) \leq Q H(\det g)^{1/2}, \quad (4)$$

and

$$H(v_1) \leq Q^2 \eta(g). \quad (5)$$

Proof. Suppose g is reduced. First, notice that (3) immediately implies (4), because $H(\det g) = H(a_1)H(a_2)$ and $H(v_1) = H(a_1)$. It thus suffices to prove (3), which is equivalent to proving that $H(\alpha) \geq 1/Q^2$. The class reduced condition already implies a lower bound on $H_{\mathbf{f}}(\alpha)$, namely: $B^{-1} \leq H_{\mathbf{f}}(\alpha)$, therefore it remains to get a lower bound on $H_{\infty}(\alpha)$. The Lovász condition implies that

$$\delta \leq H((m, \alpha)) = H_{\mathbf{f}}((m, \alpha)) H_{\infty}((m, \alpha)).$$

The right-hand side is at most

$$H_{\mathbf{f}}((m, 1)) \prod_{i=1}^d \left(|\sigma_i(m)|^2 + (e^A H_{\infty}(\alpha)^{\frac{1}{d}})^2 \right)^{\frac{1}{2}} \quad (6)$$

by the unit reduced condition and the fact that $H_{\mathbf{f}}((m, \alpha)) \leq H_{\mathbf{f}}((m, 1))$, since α is integral. Definition of size reduction implies

$$\begin{aligned} H_{\mathbf{f}}((m, 1)) \prod_{i=1}^d \left(|\sigma_i(m)|^2 + (e^A H_{\infty}(\alpha)^{\frac{1}{d}})^2 \right)^{\frac{1}{2}} &\leq \prod_{i=1}^d \left((\tau_{\sigma_i} \sigma_i(m))^2 + (\tau_{\sigma_i} e^A H_{\infty}(\alpha)^{\frac{1}{d}})^2 \right)^{\frac{1}{2}} \\ &\leq \left(\mu^2 + (C^{\frac{1}{d}} e^A H_{\infty}(\alpha)^{\frac{1}{d}})^2 \right)^{\frac{d}{2}}. \end{aligned}$$

Thus

$$\delta^{\frac{2}{d}} \leq \mu^2 + (C^{\frac{1}{d}} e^A H_{\infty}(\alpha)^{\frac{1}{d}})^2,$$

from which one can compute and deduce

$$(\delta^{\frac{2}{d}} - \mu^2)^{\frac{d}{2}} C^{-1} e^{-Ad} \leq H_{\infty}(\alpha).$$

Hence, we proved that:

$$H(\alpha) = H_{\mathbf{f}}(\alpha) H_{\infty}(\alpha) \geq B^{-1} (\delta^{\frac{2}{d}} - \mu^2)^{\frac{d}{2}} C^{-1} e^{-Ad} = \frac{1}{Q^2},$$

which implies (3). $\square$

It is likely that, in practice, we will see a much better RHF than the bound given by Proposition 2, since we lost quite a bit in the theoretical analysis above by bounding δ by (6). It is also to be reminded that $|\Delta_F|^{\frac{1}{2}} H(v_1)$ equals the determinant of a dense rank 1 module lattice of $L(g)$. Thus for comparisons with algorithms that finds a short vector, one should take the d -th root of the above bound.

3.3 Details on each routine

In this section, we fill in all the black boxes from Algorithm 1, assuming that the input g is given as a pseudobasis.

Setup We assume the knowledge of $\mathbb{Z}$ -bases $\mathcal{B} = \{w_1, \dots, w_d\}$ of $\mathcal{O}_F$, and $\mathcal{U}$ of any finite index sublattice of the unit lattice $\text{Log}(\mathcal{O}_F^*)$. The quality of these bases will affect the efficiency of our algorithm, which may be nontrivial to ensure for certain fields. However, in the practically most relevant case of $F = \mathbb{Q}(\zeta_{2^k})$, a power-of-2 cyclotomic field, there exist well-known excellent choices for both $\mathcal{B}$ and $\mathcal{U}$.

We will also need an SVP-approximation solver for lattices over $\mathbb{Z}$ as a subroutine. To this end, one is free to choose any algorithm between LLL and an exact SVP solver, or even make an adaptive choice each time. We write γ_k for the root Hermite factor for the algorithm we use on rank k lattices, that is, γ_k is a quantity satisfying

$$\|\mathbf{v}\| \leq \gamma_k^k (\det \Lambda)^{\frac{1}{k}}$$

where Λ is any rank k lattice and $\mathbf{v} \in \Lambda$ is any short vector found by that algorithm.

We follow [31] in our choice of the input module: it is given as $\mathbf{b}_1 \mathbf{v}_1 + \mathbf{b}_2 \mathbf{v}_2 \subseteq \rho(\mathcal{O}_F^2)$, where $\mathbf{b}_1, \mathbf{b}_2$ are fractional ideals of F and $\mathbf{v}_1, \mathbf{v}_2 \in \rho(\mathcal{O}_F^2)$ are linearly independent over F .

As explained previously, our module corresponds to $g \in \text{GL}(2, \mathbb{A}_F)$ where

$$g_\infty = \begin{pmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \end{pmatrix}, g_\nu = \begin{pmatrix} \pi_\nu^{-\text{ord}_\nu(\mathbf{b}_1)} & \\ & \pi_\nu^{-\text{ord}_\nu(\mathbf{b}_2)} \end{pmatrix}$$

for each $\nu \nmid \infty$. We introduce a more classical way to express g that will be easier to implement in code. We rewrite $M_\infty = g_\infty$, and for each embedding $\sigma : F \rightarrow \mathbb{C}$, we let

$$M_\sigma = \begin{pmatrix} \sigma \circ \rho^{-1}(\mathbf{v}_1) \\ \sigma \circ \rho^{-1}(\mathbf{v}_2) \end{pmatrix} = \begin{pmatrix} 1 & \\ m_\sigma & 1 \end{pmatrix} \begin{pmatrix} a_{1\sigma} & \\ & a_{2\sigma} \end{pmatrix} U_\sigma$$

where $a_{1\sigma}, a_{2\sigma} > 0$ and $U_\sigma \in K_\sigma$, and

$$M_I = \begin{pmatrix} \mathbf{b}_1^{-1} & \\ & \mathbf{b}_2^{-1} \end{pmatrix},$$

an ideal-valued matrix. Clearly M_∞ can be identified with $(M_\sigma)_{\sigma|\infty}$. It may also be helpful to construe M_I as $\prod_{\nu \nmid \infty} g_\nu$ in the arguments to follow.

Additionally, we define an auxiliary variable for our algorithm

$$M_F = \begin{pmatrix} 1 & 0 \\ m_F & 1 \end{pmatrix} \in \text{GL}(2, F),$$

where $m_F = 0$ initially. We will write m for the adele such that $m_\nu = (m_F)_\nu$ for finite ν , and, for $\sigma \mid \infty$, whose σ -component coincide with m_σ above. Similarly, for $i = 1, 2$,

a_i will be the adele for which $(a_i)_\nu = \pi_\nu^{-\text{ord}_\nu b_i}$ for finite ν , and $(a_i)_\sigma = a_{i\sigma}$ for infinite σ . Hence g has an Iwasawa decomposition

$$g = \begin{pmatrix} 1 & \\ m & 1 \end{pmatrix} \begin{pmatrix} a_1 & \\ & a_2 \end{pmatrix} \prod_{\sigma|\infty} U_\infty,$$

and M_F, M_I, M_∞ simply encode the same information in the classical language.

Algorithm 2 size-reduce

Require: Parameters μ, C , a subfield $E \subseteq F$ of degree d_E , matrices $\mathbf{M}$ and $\mathbf{M}_E$ representing bases of $\rho(\mathcal{O}_F)$ and $\rho(\mathcal{O}_E)$, an HSVP solver in dimension $d + d_E$ and input $(m_\sigma)_{\sigma:F \rightarrow \mathbb{C}}$

- 1: $\omega \leftarrow \mu C^{-1/d}$
- 2: $T \leftarrow \begin{pmatrix} \omega \mathbf{M}_E & \mathbf{M}_E \text{diag}(m_\sigma) \\ & -\mathbf{M} \end{pmatrix}$
- 3: Use the HSVP solver on T to obtain $\mathbf{x} = (q_1, \dots, q_{d_E}, p_1, \dots, p_d)$ such that $\mathbf{x}T$ is short
- 4: **return** $p \leftarrow (p_1, \dots, p_d)\mathbf{M}, q \leftarrow (q_1, \dots, q_{d_E})\mathbf{M}_E$

Size reduction We will compute $p, q \in \mathcal{O}_F$ such that

$$|q_\sigma m_\sigma - p_\sigma| \leq \mu \text{ and } |q_\sigma| \leq C^{\frac{1}{d}} \quad (7)$$

for every embedding $\sigma : F \hookrightarrow \mathbb{C}$. Let us first explain how this leads to size reduction. Write $r = p/q \in F$. We assume that the algorithm **size-reduce** has input $m = ((m_F)_\mathbf{f}, m_\infty) \in \mathbb{A}_F$ such that $m_F = 0$. It will ultimately replace m by $m - r$. Note that $H_\mathbf{f}((m - r, 1)) \leq H_\infty(q^{-1}) = H_\infty(q)$, the former since $m_\mathbf{f} = (m_F)_\mathbf{f} = 0$, and the latter by the product formula. Then, with $\tau_\sigma = |q_\sigma|$, $H_\infty(q) = \prod_{\sigma:F \hookrightarrow \mathbb{C}} \tau_\sigma$ holds by the definition of H_∞ , and moreover, (7) translates to

$$\tau_\sigma |(m - r)_\sigma| \leq \mu \text{ and } \tau_\sigma \leq C^{\frac{1}{d}},$$

satisfying all the conditions for size reducedness indeed.

It remains to show how to obtain (7). Let

$$\mathbf{M} = \begin{pmatrix} -\rho(w_1) & - \\ & \vdots \\ -\rho(w_d) & - \end{pmatrix}.$$

Similarly, for any choice of a subextension $\mathbb{Q} \subseteq E \subseteq F$ of degree d_E , let $\mathbf{M}_E$ be the matrix whose rows consist of a $\mathbb{Z}$ -basis of $\rho(\mathcal{O}_E)$. Let $\omega = \mu C^{-1/d}$, and consider the rank $d + d_E$ lattice spanned by the rows of

$$\begin{pmatrix} \omega \mathbf{M}_E & \mathbf{M}_E \text{diag}(m_\sigma) \\ & -\mathbf{M} \end{pmatrix}. \quad (8)$$

Observe that this lattice has determinant $\omega^{d_E} \Delta$, where $\Delta = \sqrt{(d/d_E)^{d_E} |\Delta_E \Delta_F|}$,³ and its vectors are of the form $(\omega \rho(q), \rho(qm_\infty - p))$ for $q \in \mathcal{O}_E, p \in \mathcal{O}_F$. Running our choice of an HSVP solver, we obtain a nonzero vector of such form of length at most

$$\lambda = \gamma_{d+d_E}^{d+d_E} \omega^{\frac{d_E}{d+d_E}} \Delta^{\frac{1}{d+d_E}}.$$

Assume $C^{1/d} \geq \lambda \omega^{-1}$, then equivalently by definition of ω , we get $\mu \geq \lambda$ and thus $|q_\sigma m_\sigma - p_\sigma| \leq \mu$ for every σ . We also have $\omega |q_\sigma| \leq \mu$, or equivalently $|q_\sigma| \leq C^{1/d}$. Note that $q = 0$ is impossible, since it would imply $\rho(p)$ has length at most $\mu < 1$, whereas it is well-known that a nonzero vector of $\rho(\mathcal{O}_F)$ has length at least $\sqrt{d}$. This proves that (7) can indeed be attained, as long as $C^{1/d}$ is chosen to be greater than or equal to

$$\mu^{-\frac{d}{d_E}} \gamma_{d+d_E}^{(d+d_E)^2/d_E} \Delta^{\frac{1}{d_E}}. \quad (9)$$

The above discussion is summarized in pseudocode form in Algorithm 2. As an example, we can take $E = F$ and $C = \mu^{-d} \gamma_{2d}^{4d^2} |\Delta_F|$; or if we take $E = \mathbb{Q}$, then $C = \mu^{-d^2} \gamma_{d+1}^{d(d+1)^2} (d |\Delta_F|)^{d/2}$ is a suitable choice.

Algorithm 3 Lovasz-test rank 2

Require: Parameter δ , and inputs m_F, M_I, M_∞

1: **if**

$$\delta N((m_F \mathcal{O}_F + \mathbf{b}_1 \mathbf{b}_2^{-1})) \leq \prod_{\sigma: F \rightarrow \mathbb{C}} |(m_\sigma, a_{2\sigma}/a_{1\sigma})|$$

return true

2: **else return false**

Lovász test If the Lovász condition

$$\delta H((a_1, 0)) \leq H((ma_1, a_2)),$$

or equivalently as a product over all embeddings,

$$\delta N(\mathbf{b}_1) \prod_{\sigma: F \hookrightarrow \mathbb{C}} |a_{1\sigma}| \leq N((m_F \mathbf{b}_1^{-1} + \mathbf{b}_2^{-1}))^{-1} \prod_{\sigma: F \hookrightarrow \mathbb{C}} |(m_\sigma a_{1\sigma}, a_{2\sigma})|,$$

or the further simplified

$$\delta N((m_F \mathcal{O}_F + \mathbf{b}_1 \mathbf{b}_2^{-1})) \leq \prod_{\sigma: F \hookrightarrow \mathbb{C}} |(m_\sigma, a_{2\sigma}/a_{1\sigma})|,$$

is fulfilled, then we terminate the algorithm. If not, we carry out the adelic swap step below.

Algorithm 4 adelic-swap rank 2

Require: $m_F = x/y$ ($x, y \in \mathcal{O}_F$), M_I, M_∞ that failed **Lovasz-test**

- 1: Compute $c \in \mathcal{O}_F$ such that $cx\mathfrak{b}_1^{-1}, cy\mathfrak{b}_2^{-1} \subseteq \mathcal{O}_F$
 - 2: $\mathfrak{d} \leftarrow cx\mathfrak{b}_1^{-1} + cy\mathfrak{b}_2^{-1}$
 - 3: Compute $u \in (\mathfrak{b}_1\mathfrak{d})^{-1}, v \in (\mathfrak{b}_2\mathfrak{d})^{-1}$ such that $cxu + cyv = 1$
 - 4: $m_F \leftarrow 0, M_I \leftarrow \begin{pmatrix} \mathfrak{d} & \\ & (\mathfrak{b}_1\mathfrak{b}_2\mathfrak{d})^{-1} \end{pmatrix}, M_\infty \leftarrow \begin{pmatrix} \rho(cy) & \\ \rho(1/(cy)) & \rho(-u) \end{pmatrix} M_\infty$
-

Adelic swap If **Lovasz-test** returns false, we swap the two rows of g , as in the original LLL algorithm. This maneuver is equivalent to swapping the rows of M_∞ , and also swapping the rows of M_F , so that

$$M_F = \begin{pmatrix} m_F & 1 \\ 1 & \end{pmatrix}, M_I = \begin{pmatrix} \mathfrak{b}_1^{-1} & \\ & \mathfrak{b}_2^{-1} \end{pmatrix}.$$

M_I here is correct to remain unchanged, since, from the adelic point of view, on each finite place ν ,

$$\begin{pmatrix} 1 & \\ 1 & \end{pmatrix} \begin{pmatrix} 1 & \\ m_\nu & 1 \end{pmatrix} \begin{pmatrix} a_{1\nu} & \\ & a_{2\nu} \end{pmatrix} = \begin{pmatrix} m_\nu & 1 \\ 1 & \end{pmatrix} \begin{pmatrix} a_{1\nu} & \\ & a_{2\nu} \end{pmatrix}. \quad (10)$$

We would like to carry out some manipulations, so as to be able to rewrite M_F as the identity element, so that our data M_F, M_I, M_∞ comes back to representing a pseudobasis for the next iteration of **size-reduce**.

Start by writing $m_F = x/y$ for $x, y \in \mathcal{O}_F$; note that this is already accomplished in the **size-reduce** step with $y = q$. Compute nonzero $c \in \mathcal{O}_F$ such that $cx\mathfrak{b}_1^{-1}, cy\mathfrak{b}_2^{-1}$ are both integral. Suppose $cx\mathfrak{b}_1^{-1} + cy\mathfrak{b}_2^{-1} = \mathfrak{d}$. Then by Theorem 1.2 of [Coh96], we can efficiently compute $u \in (\mathfrak{b}_1\mathfrak{d})^{-1}, v \in (\mathfrak{b}_2\mathfrak{d})^{-1}$ such that $cxu + cyv = 1$.

Fix $\nu \nmid \infty$. Let us write $d_\nu = \pi_\nu^{-\text{ord}_\nu \mathfrak{d}}$. Now consider the ν -adic matrix

$$\begin{pmatrix} ua_1^{-1} & cy a_1^{-1} \\ va_2^{-1} & -cx a_2^{-1} \end{pmatrix} \begin{pmatrix} d^{-1} & \\ & a_1 a_2 d \end{pmatrix} = \begin{pmatrix} ua_1^{-1} d^{-1} & cy a_2 d \\ va_2^{-1} d^{-1} & -cx a_1 d \end{pmatrix}$$

where we suppress all the ν subscripts, and temporarily tolerate the notational overlap of d with the field degree. By construction, this is a matrix of determinant -1 consisting of entries in $\mathcal{O}_\nu$, hence an element of $\text{GL}_2(\mathcal{O}_\nu)$. Therefore we can multiply this element to the right of g without changing $L(g)$. Over each $\mathcal{O}_\nu$, we would multiply it to the right of (10), and obtain

$$\begin{aligned} \begin{pmatrix} m & 1 \\ 1 & \end{pmatrix} \begin{pmatrix} a_1 & \\ & a_2 \end{pmatrix} \begin{pmatrix} ua_1^{-1} d^{-1} & cy a_2 d \\ va_2^{-1} d^{-1} & -cx a_1 d \end{pmatrix} &= \begin{pmatrix} x/y & 1 \\ 1 & \end{pmatrix} \begin{pmatrix} ud^{-1} & cy a_1 a_2 d \\ vd^{-1} & -cx a_1 a_2 d \end{pmatrix} \\ &= \begin{pmatrix} x/y & 1 \\ 1 & \end{pmatrix} \begin{pmatrix} u & cy \\ v & -cx \end{pmatrix} \begin{pmatrix} d^{-1} & \\ & a_1 a_2 d \end{pmatrix} = \begin{pmatrix} 1/(cy) & \\ u & cy \end{pmatrix} \begin{pmatrix} d^{-1} & \\ & a_1 a_2 d \end{pmatrix}. \end{aligned}$$

³ By a straightforward argument, one can show that the sublattice $\rho(\mathcal{O}_E)$ of $\rho(\mathcal{O}_F)$, where $\mathcal{O}_E$ is seen as a subset of $\mathcal{O}_F$, has determinant $\sqrt{(d/d_E)^{d_E} |\Delta_E|}$.

In the classical language, this corresponds to setting

$$M_F = \begin{pmatrix} x/y & 1 \\ 1 & \end{pmatrix} \begin{pmatrix} u & cy \\ v & -cx \end{pmatrix} = \begin{pmatrix} 1/(cy) & \\ u & cy \end{pmatrix},$$

$$M_I = \begin{pmatrix} \mathfrak{d} & \\ (\mathfrak{b}_1 \mathfrak{b}_2 \mathfrak{d})^{-1} & \end{pmatrix}.$$

Next, we multiply g from the left by

$$\begin{pmatrix} cy & \\ -u & 1/(cy) \end{pmatrix} \in \mathrm{GL}(2, F).$$

In the classical language, this turns M_F back to the identity, and multiplies M_∞ by

$$\begin{pmatrix} \rho(cy) & \\ \rho(-u) & \rho(1/(cy)) \end{pmatrix}.$$

In summary, we have made the following changes to the variables:

$$m_F \leftarrow 0, \mathfrak{b}_1 \leftarrow \mathfrak{d}^{-1}, \mathfrak{b}_2 \leftarrow \mathfrak{b}_1 \mathfrak{b}_2 \mathfrak{d}, M_\infty \leftarrow \begin{pmatrix} \rho(cy) & \\ \rho(1/(cy)) & \rho(-u) \end{pmatrix} M_\infty.$$

Algorithm 5 unit-reduce rank 2

Require: Parameter A , input M_∞ , basis $\mathcal{U}$ of a sublattice of $\mathrm{Log}(\mathcal{O}_F^*)$

Ensure: M_∞ is unit reduced with respect to A

- 1: Compute α_∞ of M_∞
 - 2: Find an aCVP solution $\mathrm{Log}(u)$ with respect to $\mathrm{Log}(\mathcal{O}_F^*)$ and target $\mathrm{Log}(\alpha_\infty) - \frac{1}{d} \log(H_\infty(\alpha_\infty)) \cdot (1, \dots, 1, 2, \dots, 2)$, by applying Babai's nearest plane algorithm using $\mathcal{U}$
 - 3: $M_\infty \leftarrow \begin{pmatrix} 1 & \\ & u^{-1} \end{pmatrix} M_\infty$
-

Unit reduction To unit reduce $\alpha_\infty \in \mathbb{A}_\infty$ such that $\alpha_\sigma \neq 0$ for all $\sigma \mid \infty$, one simply solves aCVP with respect to the target $\mathrm{Log}(\alpha_\infty) - \frac{1}{d} \log(H_\infty(\alpha_\infty)) \cdot (1, \dots, 1, 2, \dots, 2)$, the lattice spanned by $\mathcal{U}$, and the approximate factor A , using Babai's nearest plane algorithm for example. As is well-understood, the “better” the quality of $\mathcal{U}$, the lower we can set A to be. Let $u \in \mathcal{O}_F^*$ be any preimage of this aCVP. Then it is easy to see that $u^{-1}\alpha_\infty$ is unit reduced. In particular, for any embedding $\sigma : F \hookrightarrow \mathbb{C}$,

$$e^{-A} \leq \frac{|u_\sigma^{-1}\alpha_\sigma|}{H_\infty(x)^{\frac{1}{d}}} \leq e^A.$$

Algorithm 6 *scale* (Algorithm 3.2 of [31])

Require: A fractional ideal $\mathfrak{b} \subseteq F$, a vector $\mathbf{v} \in (F \otimes \mathbb{R})^n$, the “Gram-Schmidt coefficient” $\mathbf{a} \in F \otimes \mathbb{R}$, and an HSVP solver in dimension d with RHF factor γ_d

Ensure: $\mathbf{b}\mathbf{v}$ is scaled

- 1: Apply the HSVP solver to find $\mathbf{s} \in \mathbf{a}\rho(\mathfrak{b})$ such that

$$\|\mathbf{s}\| \leq \gamma_d |\Delta_F|^{1/(2d)} \left(N(\mathfrak{b}) \prod_{\sigma} \mathbf{a}_{\sigma} \right)^{1/d}$$

- 2: Write $\mathbf{s} = \mathbf{a}\mathbf{x}$ with $\mathbf{x} \in \rho(\mathfrak{b})$
 - 3: $\mathfrak{b} \leftarrow \rho^{-1}(\mathbf{x}^{-1})\mathfrak{b}$, $\mathbf{v} \leftarrow \mathbf{x}\mathbf{v}$
-

Algorithm 7 *scale+* rank 2

Require: Parameter B , inputs M_I, M_{∞} , and an HSVP solver in dimension d with RHF factor γ_d satisfying $B \geq \gamma_d^{d^2} |\Delta_F|^{1/2}$

Ensure: α is class reduced, and bitsizes of the inputs are at most $\text{poly}(\text{input}, d, \log |\Delta_F|)$

- 1: *scale*($\mathfrak{b}_1, \mathbf{v}_1, a_{1\infty}$); update M_I, M_{∞} accordingly
 - 2: *scale*($\mathfrak{b}_2 \mathfrak{b}_1^{-1}, \mathbf{v}_2, a_{2\infty}$); update M_I, M_{∞} accordingly // $\mathfrak{b}_2$ changes by $\rho^{-1}(\mathbf{x}^{-1})\mathfrak{b}_2$, $\mathfrak{b}_1$ remains unchanged
 - 3: Apply [31, Algorithm 3.3] to $\mathfrak{b}_1 \mathbf{v}_1 + \mathfrak{b}_2 \mathbf{v}_2$, and update M_I, M_{∞} accordingly
-

Scaling for bitsize control and class reduction [31, Section 3.2] introduces mechanisms for bitsize control called *scaling* and *size reducing* (in the sense of [31], different from ours). For a pseudo-basis $\mathfrak{b}_1 \mathbf{v}_1 + \mathfrak{b}_2 \mathbf{v}_2$, these algorithms together return another pseudo-basis of the same lattice $\mathfrak{b}'_1 \mathbf{v}'_1 + \mathfrak{b}'_2 \mathbf{v}'_2$ such that

- $\mathcal{O}_F \subseteq \mathfrak{b}'_i$, $2^{-d^2} |\Delta_F|^{-\frac{1}{2}} \leq N(\mathfrak{b}'_i) \leq 1$
- $\|\rho(a_{i,\infty})\| \leq 2^d |\Delta_F|^{\frac{1}{2d}} H(a_i)^{\frac{1}{d}}$
- $\mathbf{v}'_i \in \mathcal{O}_F$, $\|\mathbf{v}'_i\| \leq \sqrt{d}(8d)^d |\Delta_F| \max_i H(a_i)^{\frac{1}{d}}$

in $\text{poly}(\text{input}, d, \log |\Delta_F|)$; see [31, Algorithms 3.2 and 3.3],⁴ also the proof of [31, Theorem 3.8]. It may help to note here that, unravelling all notations, $\rho(a_{i,\infty})$ and $H(a_i)$ above coincides with r_{ii} and $\mathcal{N}(r_{ii} I_i)$ in [31], respectively.

These algorithms of [31] use LLL algorithm in dimension d as a subroutine. Using an HSVP solver of root Hermite factor γ_d , one obtains

- $\mathcal{O}_F \subseteq \mathfrak{b}'_i$, $\gamma_d^{-d^2} |\Delta_F|^{-\frac{1}{2}} \leq N(\mathfrak{b}'_i) \leq 1$

⁴ Algorithm 3.3 of [31] contains an error that would alter the module under question. In Step 3 there, instead of rounding by the ring of integers, one must round by $I_i I_j^{-1}$. We discuss this detail in Appendix B below. From the (rank 2) adelic perspective, Algorithm 3.3 of [31] can be seen as multiplying g on the left by a unipotent matrix whose lower left entry m_F satisfies $(m_F a_1 / a_2)_{\nu} \in \mathcal{O}_{\nu}$ for every $\nu \nmid \infty$.

- $\|\rho(a_{i,\infty})\| \leq \gamma_d^d |\Delta_F|^{\frac{1}{2d}} H(a_i)^{\frac{1}{d}}$

for the first two conditions instead. For reader's convenience, we reproduced the scale algorithm of [31] with this modification in Algorithm 6.

In this section, we explain how to apply these algorithms to control the bitsizes of the variables and ensure class reducedness at the same time. This is done by the **scale+** algorithm, whose pseudocode is provided in Algorithm 7. Observe that in step 2, **scale+** scales $\mathfrak{b}_2 \mathfrak{b}_1^{-1}$ instead of $\mathfrak{b}_2$. Thus α is integral since $\mathfrak{b}_2^{-1} \mathfrak{b}_1$ is, and it satisfies $\gamma_d^{-d^2} |\Delta_F|^{-1/2} \leq N(\mathfrak{b}_2 \mathfrak{b}_1^{-1}) = H_{\mathbf{f}}(\alpha)$. Hence class reduction is achieved with $B = \gamma_d^{d^2} |\Delta_F|^{1/2}$.

In addition, from the fact that $\mathfrak{b}_1^{-1}$ is integral, it follows that $\mathcal{O}_F \subseteq \mathfrak{b}_2$, and $2^{-2d^2} |\Delta_F|^{-1} \leq N(\mathfrak{b}_2) \leq 1$, and $\mathbf{v}_2 \in \rho(\mathcal{O}_F)$ hold at the end of **scale+**. We also have $\|\rho(a_{2,\infty})\| \leq \gamma_d^{2d} |\Delta_F|^{1/d} H(a_2)^{1/d}$. Hence, the second row of g stays polynomially controlled just like the first, although a bit larger than the original scale algorithm of [31].

As a side benefit, it happens that **scale+** allows to skip line 1 of Algorithm 4, since both $\mathfrak{b}_i^{-1}$ become integral as a result.

3.4 Summary

Algorithm 8 Adelic LLL algorithm rank 2 (pseudobasis version)

Require: Parameters δ, μ, A, B, C , basis $\mathcal{B}$ of $\mathcal{O}_F$, choices of HSVP solvers for the subroutines, and a pseudobasis of the input module $\mathfrak{b}_1 \mathbf{v}_1 + \mathfrak{b}_2 \mathbf{v}_2 \subseteq \rho(\mathcal{O}_F^2)$

Ensure: Data m_F, M_I, M_∞ representing a reduced element of $\mathrm{GL}(2, \mathbb{A}_F)$ for the input module

```

1:  $m_F \leftarrow 0, M_I \leftarrow \begin{pmatrix} \mathfrak{b}_1^{-1} & \\ & \mathfrak{b}_2^{-1} \end{pmatrix}, M_\infty \leftarrow \begin{pmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \end{pmatrix}$ 
2: while true do
3:   scale+( $M_I, M_\infty$ )
4:   unit-reduce( $M_\infty$ )
5:    $(p, q) \leftarrow \text{size-reduce}(\rho(m_\infty)); m_F \leftarrow -p/q, M_\infty \leftarrow \begin{pmatrix} 1 & \\ -\rho(p/q) & 1 \end{pmatrix} M_\infty$ 
6:   if (Lovasz-test( $m_F, M_I, M_\infty$ ))
7:     break loop
8:   else
9:     adelic-swap( $m_F, M_I, M_\infty$ )
10: return  $m_F, M_I, M_\infty$ 

```

Putting everything from the previous section together, we obtain Algorithm 8, the adelic LLL algorithm on pseudobases, as well as the following main result.

Theorem 1 (Rank 2 reduction). *Algorithm 8 returns the data m_F, M_I, M_∞ representing a reduced element of $\mathrm{GL}(2, \mathbb{A}_F)$ corresponding to the input module in $\mathrm{poly}(\text{input}, d, \log |\Delta_F|)$ and $\mathrm{poly}(\text{input})$ many calls to the chosen HSVP oracles.*

Proof. Proposition 1 shows that the number of loops will be $\text{poly}(\text{input})$. In each loop, **scale+** makes two calls to the dimension d HSVP oracle, and **size-reduce** makes one call to the dimension $d + d_E$ HSVP oracle. Thus all that remains to show is that the bitsize of all variables are controlled throughout each loop.

At the start of each loop, **scale+** imposes an upper bound on the bitsizes of the inputs M_I and M_∞ by $\text{poly}(\log H(a_i), d, \log |\Delta_F|)$. Hence it suffices to show

- (i) **unit-reduce**, **size-reduce**, and **adelic-swap** change the bitsizes at most by $\text{poly}(\text{input}, d, \log |\Delta_F|)$
- (ii) $\max_i H(a_i)$ stays bounded throughout the algorithm.

If we use LLL for **size-reduce**, then (i) follows because everything runs in polynomial time. If we use a better algorithm than LLL, (i) still follows because its output will be smaller than what LLL gives. As for (ii), recall that $H(a_1)$ keeps decreasing throughout the algorithm, and that $H(a_2)$ is determined by $H(a_1)H(a_2) = H(\det g)$. Since $H(a_1) \geq 1$ (recall the input is a subset of $\rho(\mathcal{O}_F^2)$ by assumption), $H(a_2) \leq H(\det g)$ always. $\square$

The reason the output of Algorithm 8 is not in pseudobasis form is that pseudobases are usually not size reduced. If desired, one can undo the last **size-reduce** step to output a pseudobasis such that $\mathbf{b}_1 \mathbf{v}_1$ satisfies the height bound of Proposition 2. See the appendix for a comparison with the rank 2 reduction of [31].

4 Adelic LLL in rank n

4.1 Conceptual version

Definition 2 (Rank n reduced basis). Fix parameters $0 < \mu, \delta, A, B, C$, as in the $n = 2$ case. We say $g \in \text{GL}(n, \mathbb{A}_F)$ is (LLL-)reduced with respect to these parameters if it has an Iwasawa decomposition $g = uak$ such that

- (i) (size reduced) For every $m = m_{kj}$, there exists $\tau_\sigma > 0$ for each of the d embeddings $\sigma : F \hookrightarrow \mathbb{C}$ such that

$$\tau_\sigma \leq C^{\frac{1}{d}}, \tau_\sigma |m_\sigma| \leq \mu \text{ and } H_f(m, 1) \leq \prod_{\sigma : F \hookrightarrow \mathbb{C}} \tau_\sigma.$$

- (ii) (Lovász condition) $\delta \leq H((m_{k+1,k}, \alpha_k))$ for all $1 \leq k < n$.
- (iii) (unit reduced) The entries of

$$\text{Log}(\alpha_k) - \frac{1}{d} \log H_\infty(\alpha_k) \cdot (\underbrace{1, \dots, 1}_{\text{real}}, \underbrace{2, \dots, 2}_{\text{complex}})$$

are bounded by A for all $1 \leq k < n$.

- (iv) (class reduced) α_k is integral and $B^{-1} \leq H_f(\alpha_k)$ for all $1 \leq k < n$.

Combining the ideas from the classical LLL [32] and the rank 2 case, every aspect of the algorithm generalizes in a straightforward way. For example, see Algorithm 9, and the following generalizations of Propositions 1 and 2; the proofs are similar, and are delegated to Appendix A.

Algorithm 9 Adelic LLL algorithm (conceptual version)

Require: Parameters δ, μ, A, B, C and $g = uak \in \mathrm{GL}(n, \mathbb{A}_F)$.

Ensure: LLL-reduced $g \in \mathrm{GL}(n, \mathbb{A}_F)$.

```
1:  $k \leftarrow 1$ 
2: while  $k < n$  do
3:   class-reduce  $\alpha_k$ 
4:   unit-reduce  $\alpha_k$ 
5:   size-reduce  $(k+1)$ -st row of  $g$ 
6:   if  $(\mathrm{Lovasz}(k, g))$ 
7:      $k \leftarrow k + 1$ 
8:   else
9:     swap rows  $k, k+1$  of  $g$ 
10:     $k \leftarrow \max(1, k-1)$ 
11: return  $g$ 
```

Proposition 3 (Rank n termination). Define $E(g) = \prod_{h=1}^{n-1} H(v_1 \wedge \cdots \wedge v_h)$, where v_k is the k -th row vector of g . Suppose

$$\kappa = \min\{E(g') : g' \in \mathrm{GL}(n, F)g \prod_{\nu \nmid \infty} \mathrm{GL}(n, \mathcal{O}_\nu)\} > 0.$$

Then Algorithm 9 takes at most $(\log E(g) - \log \kappa)/(-\log \delta)$ swaps to terminate.

It follows from well-known facts in the geometry of numbers that $\kappa > 0$ for every $g \in \mathrm{GL}(n, \mathbb{A}_F)$ — see e.g., [3, p.18] and [33, Theorem 3.1.3]. An explicit bound such as Lemma 5 can also be obtained.

Proposition 4 (Rank n quality). Let Q be as in Proposition 2, and let $g \in \mathrm{GL}(n, \mathbb{A}_F)$ be reduced, and let v_1 denote its first row vector. Then

$$\begin{aligned} H(v_1) &\leq Q^{n-1} H(\det g)^{\frac{1}{n}}. \\ H(v_1) &\leq Q^{2(n-1)} \eta(g) \end{aligned}$$

Remark 2. Observe that, if we choose $F = \mathbb{Q}$ and $A = 0, B = 1, C = 1$, then we get $Q = (\delta^2 - \mu^2)^{-1/4}$. Therefore Proposition 4 recovers the well-known RHF upper bound for the original LLL algorithm.

4.2 Implementable pseudobasis version

We follow much of the notations and conventions from the previous section on the $n = 2$ case. We let $\mathcal{B}$ and $\mathcal{U}$ be as earlier, and assume the input module is given in the integral pseudobasis form $\mathbf{b}_1 \mathbf{v}_1 + \cdots + \mathbf{b}_n \mathbf{v}_n \subseteq \rho(\mathcal{O}_F^n)$ with $\mathbf{v}_1, \dots, \mathbf{v}_n \in \rho(\mathcal{O}_F^n)$. The matrices $M_F, M_I, M_\infty = (M_\sigma)_{\sigma \mid \infty}, g$ are defined in the analogous manner, as well as the notations referring to their entries, except now that M_F has multiple nontrivial entries

Algorithm 10 adelic-swap

Require: $k, D = (M_F, M_I, M_\infty, M'_\infty)$ that failed **Lovasz-test**

- 1: Compute $x, y \in \mathcal{O}_F$ such that $(m_F)_{k+1,k} = x/y$
 - 2: Compute $c \in \mathcal{O}_F$ such that $cx\mathfrak{b}_k^{-1}, cy\mathfrak{b}_{k+1}^{-1} \subseteq \mathcal{O}_F$
 - 3: $\mathfrak{d} \leftarrow cx\mathfrak{b}_k^{-1} + cy\mathfrak{b}_{k+1}^{-1}$
 - 4: Compute $u \in (\mathfrak{b}_k\mathfrak{d})^{-1}, v \in (\mathfrak{b}_{k+1}\mathfrak{d})^{-1}$ such that $cxu + cyv = 1$
 - 5: $M'_\infty \leftarrow B_k \begin{pmatrix} \rho(u) & \rho(cy) \\ \rho(v) & \rho(-cx) \end{pmatrix}^{-1} M'_\infty$
 - 6: $M_F \leftarrow B_k \begin{pmatrix} & cy \\ 1/(cy) & -u \end{pmatrix} \cdot M_F \cdot B_k \begin{pmatrix} u & cy \\ v & -cx \end{pmatrix}$
 - 7: $M_I \leftarrow B_k \begin{pmatrix} \mathfrak{b}_k\mathfrak{d} & \\ & (\mathfrak{b}_k\mathfrak{d})^{-1} \end{pmatrix} M_I$
 - 8: $M_\infty \leftarrow B_k \begin{pmatrix} & \rho(cy) \\ \rho(1/(cy)) & \rho(-u) \end{pmatrix} M_\infty$
-

Algorithm 11 clean

Require: $k, D = (M_F, M_I, M_\infty, M'_\infty)$

Ensure: $(m_F)_{h,h'} = 0$ for all $h = k$ and $k+1, h' < h$

- 1: $\mathbf{v}_k \leftarrow \mathbf{v}'_k, \mathbf{v}_{k+1} \leftarrow \mathbf{v}'_{k+1}$
 - 2: $(m_F)_{h,h'} \leftarrow 0$ for all $h = k$ and $k+1, h' < h$
-

Algorithm 12 Adelic LLL algorithm (pseudobasis version)

Require: Parameters δ, μ, A, B, C , basis $\mathcal{B}$ of $\mathcal{O}_F$, choices of HSVP solvers for the subroutines, and a pseudobasis of the input module $\mathfrak{b}_1\mathbf{v}_1 + \dots + \mathfrak{b}_n\mathbf{v}_n \subseteq \rho(\mathcal{O}_F^n)$.

Ensure: Data $D = (M_F, M_I, M_\infty, M'_\infty)$ representing a reduced element of $\text{GL}(n, \mathbb{A}_F)$ for the input module.

- 1: $M_F \leftarrow \text{Id}_n, M_I \leftarrow \text{diag}(\mathfrak{b}_1^{-1}, \dots, \mathfrak{b}_n^{-1}), M_\infty \leftarrow (\mathbf{v}_1^T \cdots \mathbf{v}_n^T)^T, M'_\infty \leftarrow M_\infty$
 - 2: $k \leftarrow 1$
 - 3: **while** $k < n$ **do**
 - 4: **if** $(k = 1)$ **scale** $(\mathfrak{b}_1, \mathbf{v}_1, a_{1\infty}), \mathbf{v}'_1 \leftarrow \mathbf{v}_1$
 - 5: **scale** $(\mathfrak{b}_{k+1}\mathfrak{b}_k^{-1}, \mathbf{v}_{k+1}, a_{k+1,\infty})$
 - 6: **unit-reduce** row $k+1$
 - 7: $\mathbf{v}'_{k+1} \leftarrow \mathbf{v}_{k+1}$
 - 8: Apply [31, Algorithm 3.3] to row $k+1$; update M_∞, M'_∞ accordingly
 - 9: **size-reduce** $m_{k+1,k}, \dots, m_{k+1,1}$; update M_F, M_∞ accordingly
 - 10: **if** (**Lovasz-test** at row k)
 - 11: $k \leftarrow k+1$
 - 12: **else**
 - 13: **adelic-swap** rows k and $k+1$
 - 14: **clean** rows $k, k+1$
 - 15: $k \leftarrow \max(1, k-1)$
 - 16: **return** D
-

we refer to its (k, j) -entry by $(m_F)_{kj}$. Additionally, we define $M'_\infty = \rho(M_F^{-1})M_\infty$, and refer to its k -th row as $\mathbf{v}'_k$; we will need this for bitsize control.

Algorithms 10 to 12 provide a complete pseudocode description of adelic LLL for rank n module lattices, with Algorithm 12 being the main algorithm. We omitted the descriptions of **unit-reduce** and **Lovasz-test** for brevity, as they are straightforward generalizations of their rank 2 counterparts. Indeed, as with the classical LLL, it operates on one 2×2 block at a time, and the operation is no different than the rank 2 adelic LLL.

A few notations are introduced to pinpoint where the algorithm is operating. In Algorithm 10, $B_k(M)$ for a 2×2 matrix M denotes the matrix constructed by first taking the $n \times n$ identity matrix and then replacing the 2×2 block matrix starting at (k, k) -entry by M . In Algorithm 11, $E_{kj}(x)$ is the elementary matrix constructed by first taking the $n \times n$ identity matrix and then taking its (k, j) -entry to be x . In addition, line 5 of Algorithm 10, and the correctness of Algorithm 11, are both explained by the relation $M'_\infty = \rho(M_F^{-1})M_\infty$.

A note on bitsize control We continue to employ the bitsize control method of [31]. One difference from the rank 2 case is that, while in Algorithm 8 the entire data at the start and the end of the loop is in pseudobasis form, in the general case it is not. This is why we use the additional bookkeeping variable M'_∞ , whose i -th row vector we denote by $\mathbf{v}'_i$. The first k rows of g span the same module lattice as $\mathbf{b}_1\mathbf{v}'_1 + \dots \mathbf{b}_k\mathbf{v}'_k$, and this is what we use to apply (the corrected version of) [31, Algorithm 3.3] in line 8 of Algorithm 12. Similarly as in the proof of Theorem 1, we obtain the following.

Theorem 2 (Rank n reduction). *Algorithm 12 returns the data $(M_F, M_I, M_\infty, M'_\infty)$ representing a reduced element of $\text{GL}(n, \mathbb{A}_F)$ corresponding to the input module in $\text{poly}(\text{input}, d, \log |\Delta_F|)$ and $\text{poly}(\text{input})$ many calls to the chosen HSVP oracles.*

4.3 Theoretical performance of adelic LLL

Following [34], we define (γ, n) -ModuleHSVP (resp. (γ, n) -ModuleSVP) to be the problem of finding, given a rank n module lattice L over a field F of degree d , a nonzero vector of L of length at most $\gamma(\det L)^{1/(nd)}$ (resp. $\gamma\lambda_1(L)$). We also refer to $(\gamma, 1)$ -ModuleHSVP as γ -IdealHSVP.

Let $0 < \delta, \mu < 1$, A, B, C be the parameters chosen for adelic LLL (Algorithm 8). Recall

$$Q = (\delta^{\frac{2}{d}} - \mu^2)^{-\frac{d}{4}} (CB)^{\frac{1}{2}} e^{\frac{Ad}{2}}.$$

Proposition 4 establishes that adelic LLL, applied to a rank n module lattice $L = L(g)$, returns a rank 1 submodule $\Lambda \subseteq L$ of height at most $Q^{n-1}H(\det g)^{1/n}$ (resp. $Q^{2(n-1)}\eta(g)$), or equivalently, of determinant at most $Q^{n-1}(\det L)^{1/n}$ (resp. $\Delta_F^{1/2}Q^{2(n-1)}\eta(g)$). Hence, if one solves γ -IdealHSVP, then one finds a vector of Λ of length at most $\gamma \cdot Q^{(n-1)/d}(\det L)^{1/(nd)}$ (resp. $\gamma \cdot \Delta_F^{1/(2d)}Q^{2(n-1)/d}\eta(g)^{1/d}$), then trivially one also finds a vector of L of length at most $\gamma \cdot Q^{(n-1)/d}(\det L)^{1/(nd)}$ (resp. $\gamma \cdot \Delta_F^{1/(2d)}Q^{2(n-1)/d}d^{-1/2}\lambda_1(L)$ by Lemma 3). This proves

Theorem 3. *Fix a number field F , and let Q be as above. Then there exists a reduction from $(\gamma Q^{(n-1)/d}, n)$ -ModuleHSVP and $(\gamma \Delta_F^{1/(2d)} Q^{2(n-1)/d} d^{-1/2}, n)$ -ModuleSVP to γ -IdealHSVP, of $\text{poly}(\text{input}, d, \log |\Delta_F|)$ time complexity and $\text{poly}(\text{input})$ calls to the chosen HSVP oracle.*

Naturally one would be interested in the size of Q , since Theorem 3 is most meaningful when γ is much greater than $Q^{(n-1)/d}$ (and when n is small relative to d). One can choose $\delta = 0.999$ and $\mu = 0.5$ as in the classical LLL. As discussed earlier, one can take $C = \mu^{-d} \gamma_{2d}^{4d^2} |\Delta_F|$ (taking $E = F$ in Algorithm 2) and $B = \gamma_d^{d^2} |\Delta_F|^{1/2}$.

For a further discussion, let us restrict to the case $F = \mathbb{Q}(\zeta_{2^{\kappa+1}})$ ($\kappa \geq 2$), so that $d = 2^\kappa$. In this case, $|\Delta_F| = d^d$. In addition, from [8, Corollary 6.4], one can take $A = O(\sqrt{d \log d})$. Hence we obtain

$$Q^{\frac{1}{d}} = \gamma_{2d}^{2d} \gamma_d^{d/2} e^{O(\sqrt{d \log d})}.$$

If we use the slide algorithm [17] or SDBKZ [27] of blocksize β for the subroutine, for which rigorous performance estimates are available, then $\gamma_d^d \leq \beta^{d/(2\beta)}$, and likewise $\gamma_{2d}^{2d} \leq \beta^{d/\beta}$. Taking $\beta = \sqrt{d}$ for example, we have

$$Q^{\frac{1}{d}} \leq e^{\frac{5}{8} \sqrt{d} \log d \cdot (1+o_d(1))}.$$

Therefore, for instance, if $n \leq \log d$ and $\gamma \geq e^{\sqrt{d}(\log d)^3}$, then Theorem 3 yields a $e^{O(\sqrt{d})}$ -time reduction from $(\gamma^{1+o_d(1)}, n)$ -ModuleHSVP to γ -IdealHSVP.

One may also be interested in the following comparison. Suppose $\beta \leq \sqrt{d}$. Applying the slide algorithm or SDBKZ of blocksize β directly to a rank n module lattice L , one obtains a vector of length $\approx \beta^{nd/(2\beta)} \cdot (\det L)^{1/(nd)}$. On the other hand, if we apply adelic LLL using the same algorithm as the SVP oracle, and apply the same oracle to the resulting rank 1 submodule as well, then one obtains a vector of length $\approx \beta^{d/(2\beta)} \cdot \beta^{2.5nd/(2\beta)} \cdot (\det L)^{1/(nd)}$ up to terms of lesser size, by the formula for $Q^{1/d}$ above. In this sense, it may be said that the current version of adelic LLL is asymptotically “2.5 times worse”, since, for large n and d , β needs to be about a factor of 2.5 greater to match the output quality of BKZ- β . This loss mainly comes from the size of the parameter C , which measures how well one can solve the Diophantine approximation problem associated to the size reducedness. This motivates the following question.

Question 1. Could **size-reduce** (Algorithm 2) be improved to lower the size of the required C , without increasing the time complexity? In particular, would it be possible to exploit the field structure to achieve this?

This is not entirely implausible, in light of the following observation.

Proposition 5. *If $C = \mu^{-d} |\Delta_F|$, then there exists $p, q \in \mathcal{O}_F$ satisfying (7) for any given $m_\infty \in \mathbb{A}_\infty$.*

Proof. Consider the lattice Λ spanned by the rows of (8), with $E = F$. This is a rank $2d$ lattice with $\det \Lambda = \omega^d |\Delta_F|$. Therefore, by Minkowski’s first theorem, there exists

a nonzero vector of this lattice in the box $[-\mu^{-1}\omega|\Delta_F|^{1/d}, \mu^{-1}\omega|\Delta_F|^{1/d}]^d \times [-\mu, \mu]^d$, since this box has volume $2^{2d}\omega^d|\Delta_F| = 2^{2d}\det \Lambda$. Recall that every vector of Λ is of the form $(\omega\rho(q), \rho(qm_\infty - p))$; the elements $p, q \in \mathcal{O}_F$ corresponding in this manner to the vector in the box is clearly seen to satisfy (7) with $C = \mu^{-d}|\Delta_F|$. $\square$

This statement suggests that the extra factor $\gamma_{2d}^{4d^2}$ in our choice of C does not reflect some intrinsic limitation of adelic LLL, but rather a limitation of the current method of size reduction.

If one can find a way to reduce the exponent on γ_{2d} by a factor of half or more, then there is hope for adelic LLL to theoretically outperform the standard reduction algorithms in practically important cases. The loss from the size of B can be circumvented by other methods. For instance, since it is widely believed that the maximal totally real subextension of $\mathbb{Q}(\zeta_{2^k+1})$ has class number one — under the buzzword *Weber's class number problem*; see for instance [28] — using the method of [2] it is possible to set $B = 1$ for modules over a field of such kind.

Remark 3. One may be interested in making the above comparisons in terms of the heights of the rank 1 submodules, rather than the lengths of the lattice vectors as we have done here. It turns out that doing so one observes essentially the identical quantitative behavior.

To elaborate, continue with the earlier assumptions, and suppose one applies BKZ- β to find a vector $\mathbf{v}$ of L . By [23, Lemma 2.2] for instance, the rank 1 submodule $\mathcal{O}_F \cdot \mathbf{v}$ has height at most

$$d^{-d/2}\|\mathbf{v}\|^d \leq d^{-d/2}\beta^{\frac{nd^2}{2\beta}} \cdot (\det L)^{\frac{1}{n}}.$$

Comparing with the height bound of adelic LLL

$$Q^{n-1}(\det L)^{\frac{1}{n}} \approx \beta^{2.5nd^2/(2\beta)} \cdot (\det L)^{\frac{1}{n}},$$

one again notices that, asymptotically in n and d , the latter is 2.5 times worse.

References

1. H. Bambury. On structured lattices in cryptology. Ph.D. Thesis, ENS-PSL.
2. J.-F. Biasse and F. Song. Efficient quantum algorithms for computing class groups and solving the principal ideal problem in arbitrary degree number fields. In *Proceedings of the 27th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 893-902. SIAM, January 2016.
3. E. Bombieri and J. Vaaler. On Siegel’s lemma. *Inventiones Mathematicae*, 73:11–32, 1983.
4. A. Borel. *Introduction to Arithmetic Groups*. University Lecture Series, volume 73. American Mathematical Society, Providence, RI, 2019.
5. K. de Boer, L. Ducas, A. Pellet-Mary, and B. Wesolowski. Random self-reducibility of Ideal-SVP via Arakelov random walks. In *Advances in Cryptology—CRYPTO 2020*, LNCS 12171, pages 243–273. Springer, 2020.
6. K. de Boer, J. Felderhoff. Quantumly Computing S-Unit Groups in Quantified Polynomial Time and Space. In *Advances in Cryptology—CRYPTO 2026*.
7. K. de Boer, A. Page, R. Toma, and B. Wesolowski. Average hardness of SIVP for module lattices of fixed rank. To appear in STOC.
8. R. Cramer, L. Ducas, C. Peikert, and O. Regev. Recovering short generators of principal ideals in cyclotomic rings. In M. Fischlin and J.-S. Coron, editors, *Advances in Cryptology - EUROCRYPT 2016*, pages 559-585, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
9. H. Cohen. Hermite and Smith normal form algorithms over Dedekind domains. *Math. Comp.*
10. G. De Micheli, D. Micciancio, A. Pellet-Mary, and N. Tran. Reductions from Module Lattices to Free Module Lattices, and Application to Dequantizing Module-LLL. In: Handschuh, H., Lysyanskaya, A. (eds) *Advances in Cryptology - CRYPTO 2023*. Lecture Notes in Computer Science, vol 14085. Springer, Cham.
11. L. Ducas, L. Engelberts, and P. de Perthuis. Predicting Module-Lattice Reduction. *Advances in Cryptology - ASIACRYPT 2025: 31st International Conference on the Theory and Application of Cryptology and Information Security*, Melbourne, VIC, Australia, December 8-12, 2025, *Proceedings*, Part III Pages 133-166
12. S. Dözlü and J. Krämer. Application of automorphic forms to lattice problems. *J. Math. Crypto*. <https://eprint.iacr.org/2022/742.pdf>
13. T. Espitau, P. Kirchner, and P.-A. Fouque. Algebraic and euclidean lattices: optimal lattice reduction and beyond. *arXiv:1912.04586*
14. C. Fieker and M. Pohst. On lattices over number fields. In: Cohen, H. (eds) *Algorithmic Number Theory*. Lecture Notes in Computer Science, vol 1122. Springer, Berlin, Heidelberg.
15. E. Bayer-Fluckiger, E. and G. Nebe. On the Euclidean minimum of some real number fields. *Journal de Théorie des Nombres de Bordeaux*, 17, 437-454.
16. N. Gama, N. Howgrave-Graham, and P. Q. Nguyen. Symplectic lattice reduction and NTRU. In S. Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of LNCS, pages 233-253. Springer, 2006.
17. N. Gama and P. Q. Nguyen. Finding short lattice vectors within Mordell’s inequality. *STOC 2008*.
18. Y. H. Gan, C. Ling, and W. H. Mow. Complex lattice reduction algorithm for low-complexity full-diversity MIMO detection. *IEEE Trans Signal Processing*, 2009.
19. N. Gargava, V. Serban, M. Viazovska, and I. Viglino. Module lattices and their shortest vectors. *arXiv*
20. P. Garrett. *Modern analysis of automorphic forms by example*, Vol 1. Cambridge studies in advanced mathematics 173, Cambridge University Press, 2018.

21. A. Karenin and E. Kirshanova. Finding Dense Submodules with Algebraic Lattice Reduction. In: Vaudenay, S., Petit, C. (eds) Progress in Cryptology - AFRICACRYPT 2024. Lecture Notes in Computer Science, vol 14861. Springer, Cham.
22. T. Kim and C. Lee. Lattice reductions over euclidean rings with applications to cryptanalysis. In IMACC, 2017.
23. S. Kim. Adelic Rogers integral formula. J. London Math. Soc., 109: e12830.
24. P. Kirchner, T. Espitau, and P.-A. Fouque. Fast reduction of algebraic lattices over cyclotomic fields. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology - CRYPTO 2020. pp. 155-185 (2020)
25. S. Lyu, C. Porter, and C. Ling. Lattice reduction over imaginary quadratic fields. IEEE Transactions on Signal Processing, vol. 68, 6380-6393.
26. D. Micciancio. The hardness of the closest vector problem with preprocessing. IEEE Transactions on Information Theory, 47(3), 1212-1215.
27. D. Micciancio and M. Walter. Practical, predictable lattice basis reduction. In Advances in Cryptology - EUROCRYPT 2016, Part I, edited by Marc Fischlin and Jean-Sébastien Coron, LNCS Vol. 9665, pp. 820-849. Springer, Heidelberg, May 2016.
28. J. Miller. Class numbers of totally real number fields. Ph.D. Thesis, Rutgers University.
29. National Institute of Standards and Technology. NIST Internal Report 8413. <https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8413-upd1.pdf>
30. S. Kim. Adelic Rogers integral formula, J. Lond. Math. Soc. (2) 109 (2024), no. 1.
31. C. Lee, A. Pellet-Mary, D. Stehlé, and A. Wallet. An LLL algorithm for module lattices. In *Advances in Cryptology—ASIACRYPT 2019*. Springer, 2019.
32. A. K. Lenstra, H. W. Lenstra, Jr., and L. Lovász. Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261:515–534, 1982.
33. M. Morishita and T. Watanabe. Adelic geometry of numbers. Advanced Studies in Pure Mathematics 30, 2001, Class Field Theory — Its Centenary and Prospect, pp. 509-536
34. T. Mukherjee and N. Stephens-Davidowitz. Lattice reduction for modules, or how to reduce ModuleSVP to ModuleSVP. In *Advances in Cryptology—CRYPTO 2020*, LNCS 12171, pages 213–242. Springer, 2020.
35. H. Napias. A generalization of the LLL-algorithm over Euclidean rings or orders. *Journal de Théorie des Nombres de Bordeaux*, 8(2):387–396, 1996.
36. A. Pellet-Mary and G. Hanrot and D. Stehlé Approx-SVP in Ideal Lattices with Preprocessing. In *Advances in Cryptology—EUROCRYPT 2019*. Springer, 2019.
37. B. Romano and J. Thorne. An LLL algorithm with symmetries. Int. J. Number Theory, Vol. 21, No. 06, pp. 1361-1393 (2025).
38. D. Simon. Selected applications of LLL in number theory. Chapter 7 in The LLL Algorithm, Survey and applications, Information Security and Cryptography, Nguyen and Vallée (eds), Springer Verlag Berlin Heidelberg (2010), 265-282.
39. F. Strömberg. A reduction algorithm for Hilbert modular groups. *Journal of Number Theory*, 241:581–602, 2022.
40. J. Thunder. Asymptotic estimates for rational points of bounded height on flag varieties. *Compositio Math.* 88 (1993), no. 2, 155-186.
41. J. L. Thunder. Higher-dimensional analogs of Hermite’s constant,” *Michigan Mathematical Journal* 45 (1998), no. 2, 301–314.
42. L. C. Washington. *Introduction to Cyclotomic Fields*. Graduate Texts in Mathematics, volume 83. Springer, second edition, 1997.
43. A. Weil. *Basic Number Theory*. Springer-Verlag, New York, 1974.

Supplementary materials

A Proof of statements regarding rank n adelic LLL

Proof (Proposition 3). Observe $H(v_1 \wedge \cdots \wedge v_h) = H(a_1) \cdots H(a_h)$. When v_k and v_{k+1} are swapped, $H(v_1 \wedge \cdots \wedge v_h)$ changes only if $h = k$; and in this case, it changes to

$$H(a_1) \cdots H(a_{k-1}) H((m_{k+1,k} a_k, a_{k+1})).$$

Since the Lovász condition failed, we have

$$H((m_{k+1,k} a_k, a_{k+1})) < \delta H(a_k),$$

which causes $\log E(g)$ to decrease by at least $-\log \delta$. But by assumption $\log E(g)$ cannot be smaller than $\log \kappa$. This completes the proof. $\square$

Proof (Proposition 4). By applying Proposition 2 to 2×2 diagonal blocks of the ua part of g , we have

$$H(a_i)^{\frac{1}{2}} \leq Q H(a_{i+1})^{\frac{1}{2}}$$

for every i , and consequently

$$H(a_i)^{\frac{1}{2}} \leq Q^j H(a_{i+j})^{\frac{1}{2}}$$

for every j as well. Thus

$$\begin{aligned} H(a_1)^{\frac{n-1}{2}} &\leq Q^{1+\dots+(n-1)} H(a_2)^{\frac{1}{2}} \cdots H(a_n)^{\frac{1}{2}} \\ \Rightarrow H(a_1)^{\frac{n}{2}} &\leq Q^{\frac{(n-1)n}{2}} H(\det g)^{\frac{1}{2}} \\ \Rightarrow H(a_1) &\leq Q^{n-1} H(\det g)^{\frac{1}{n}} \end{aligned}$$

as desired. The final inequality follows from Lemma 4. $\square$

B A correction to Algorithm 3.3 in [31]

In this section, we follow the notation of [31].

Definition 3 (Rounding with respect to a fractional ideal). Let $J \subset K$ be a nonzero fractional ideal, viewed as a lattice in $K_{\mathbb{R}}$ via the canonical embedding. For any $y \in K_{\mathbb{R}}$, we define

$$\text{round}_J(y) \in J$$

to be an element obtained by applying Babai's nearest-plane algorithm to y with respect to a chosen LLL-reduced $\mathbb{Z}$ -basis of J .

In particular, for indices $i < j$, we define the size reduction coefficient by

$$x_{ij} := \text{round}_{I_i I_j^{-1}} \left(\frac{r_{ij}}{r_{ii}} \right) \in I_i I_j^{-1}.$$

Lemma 6 (Module preservation under ideal-compatible column reduction).

Let

$$\mathcal{B} = ((I_1, b_1), \dots, (I_n, b_n))$$

be a pseudo-basis of an R -module $M \subset K_{\mathbb{R}}^m$. Fix distinct indices $i < j$, and let $x \in I_i I_j^{-1}$. Define a new family

$$\mathcal{B}' = ((I_1, b_1), \dots, (I_j, b_j - x b_i), \dots, (I_n, b_n)),$$

where all pairs except the j -th one remain unchanged.

Then $\mathcal{B}'$ is again a pseudo-basis of the same module M . Equivalently,

$$\sum_{k=1}^n I_k b_k = \sum_{k \neq j} I_k b_k + I_j (b_j - x b_i).$$

Proof. Let

$$M = \sum_{k=1}^n I_k b_k \quad \text{and} \quad M' = \sum_{k \neq j} I_k b_k + I_j (b_j - x b_i).$$

Since $x \in I_i I_j^{-1}$, we have

$$x I_j \subseteq I_i.$$

Therefore,

$$I_j (b_j - x b_i) \subseteq I_j b_j + I_i b_i \subseteq M,$$

which implies $M' \subseteq M$.

Conversely, from

$$b_j = (b_j - x b_i) + x b_i,$$

we obtain

$$I_j b_j \subseteq I_j (b_j - x b_i) + x I_j b_i.$$

Using again the inclusion $x I_j \subseteq I_i$, we get $x I_j b_i \subseteq I_i b_i$. Hence

$$I_j b_j \subseteq I_j (b_j - x b_i) + I_i b_i \subseteq M'.$$

Since $I_k b_k \subseteq M'$ for all $k \neq j$, it follows that $M \subseteq M'$.

Therefore $M = M'$, as claimed. $\square$

Corollary 1. *With the above definition of rounding, if one sets*

$$x_{ij} = \text{round}_{I_i I_j^{-1}} \left(\frac{r_{ij}}{r_{ii}} \right),$$

then the update

$$b_j \leftarrow b_j - x_{ij} b_i$$

preserves the module spanned by the pseudo-basis.

To justify the modified size reduction step algorithmically, we fix the bit-length convention and the norm on $K_{\mathbb{R}}$ used throughout this appendix.

We assume that the number field K is given by a defining polynomial $f \in \mathbb{Q}[X]$, so that $K \simeq \mathbb{Q}[X]/(f)$, and that elements of K are represented by polynomials of degree $< d$ with rational coefficients. The bit-length of an element of K is defined to be the total bit-length of the numerators and denominators of these coefficients written in lowest terms. We also assume that a fixed $\mathbb{Z}$ -basis $B = (w_1, \dots, w_d)$ of the ring of integers R is given.

Archimedean quantities in $K_{\mathbb{R}}$ are not assigned a bit-length directly. Instead, when such quantities arise from exact elements of K , their size is controlled by the bit-length of the exact element together with Archimedean norm bounds. For $x = (x_{\sigma})_{\sigma} \in K_{\mathbb{R}}$, we denote by $\|x\|$ the Euclidean norm induced by the canonical embedding, namely

$$\|x\|^2 = \sum_{\sigma: K \hookrightarrow \mathbb{C}} |x_{\sigma}|^2,$$

where the sum ranges over all d embeddings, so each conjugate pair of complex embeddings is counted twice. If $x \in K$, then $x_{\sigma} = \sigma(x)$.

We use the absolute norm $N(\cdot)$ on nonzero fractional ideals, extended multiplicatively from integral ideals. In particular, if $A \subseteq B$ are nonzero fractional ideals, then $N(A) \geq N(B)$. We shall also use the following notation for Archimedean scalings of fractional ideals: for $a = (a_{\sigma})_{\sigma} \in K_{\mathbb{R}}^{\times}$ and a nonzero fractional ideal I , set

$$N(aI) := N(I) \prod_{\sigma: K \hookrightarrow \mathbb{C}} |a_{\sigma}|.$$

Thus, in particular, $N(r_{ii}I_i)$ denotes this quantity when r_{ii} is a nonzero Gram–Schmidt diagonal coefficient.

With this notation, we define a scaled pseudo-basis as follows.

Definition 4. A pseudo-basis $((I_i, b_i))_{i \leq n}$, with $I_i \subset K$ and $b_i \in K^m$, viewed in $K_{\mathbb{R}}^m$ via the canonical embedding, is said to be scaled if for all $i \leq n$,

$$R \subseteq I_i, \quad N(I_i) \geq 2^{-d^2} \Delta_K^{-1/2}, \quad \text{and} \quad \|r_{ii}\| \leq 2^d \Delta_K^{1/(2d)} N(r_{ii}I_i)^{1/d}.$$

Lemma 7. Let $\mathcal{B} = ((I_i, b_i))_{i \leq n}$ be a scaled pseudo-basis of a module $M \subset K^m$. For each pair $i < j$, set $J_{ij} := I_i I_j^{-1}$, and let

$$x_{ij} := \text{round}_{J_{ij}} \left(\frac{r_{ij}}{r_{ii}} \right) \in J_{ij}$$

be obtained by Babai’s nearest-plane algorithm with respect to an LLL-reduced $\mathbb{Z}$ -basis of J_{ij} . Update

$$b_j \leftarrow b_j - x_{ij} b_i$$

for $j = 2, \dots, n$, and for each fixed j , for $i = j - 1, j - 2, \dots, 1$.

Then the bit-lengths of all vectors b_j remain bounded by

$$\text{poly}(L, n, d, \log \Delta_K).$$

Proof. For each pair $i < j$, let $(u_{ij,1}, \dots, u_{ij,d})$ be the chosen LLL-reduced $\mathbb{Z}$ -basis of $J_{ij} = I_i I_j^{-1}$, and let

$$e_{ij} := \frac{r_{ij}}{r_{ii}} - x_{ij}.$$

Since x_{ij} is obtained by Babai rounding, the remainder e_{ij} belongs to the fundamental parallelepiped of $(u_{ij,t})_t$. Hence

$$\|e_{ij}\| \leq \frac{1}{2} \sum_{t=1}^d \|u_{ij,t}\| \leq d \max_t \|u_{ij,t}\|.$$

Now, because the pseudo-basis is scaled, we have $N(I_\ell) \leq 1$ and $N(I_\ell) \geq 2^{-d^2} \Delta_K^{-1/2}$ for every ℓ . Therefore

$$N(J_{ij}) = \frac{N(I_i)}{N(I_j)} \leq 2^{d^2} \Delta_K^{1/2}.$$

Since the basis $(u_{ij,1}, \dots, u_{ij,d})$ of J_{ij} is LLL-reduced, we have

$$\|u_{ij,t}\| \leq 2^{\frac{d-1}{2}} \|u_{ij,t}^*\| \quad \text{for all } t.$$

Taking the product over all t yields

$$\prod_{t=1}^d \|u_{ij,t}\| \leq 2^{\frac{d(d-1)}{2}} \prod_{t=1}^d \|u_{ij,t}^*\|.$$

Since

$$\prod_{t=1}^d \|u_{ij,t}^*\| = \Delta_K^{1/2} N(J_{ij}),$$

it follows that

$$\prod_{t=1}^d \|u_{ij,t}\| \leq 2^{\frac{d(d-1)}{2}} \Delta_K^{1/2} N(J_{ij}).$$

We now bound $\max_t \|u_{ij,t}\|$. For this, we use the inclusion part of the scaledness assumption. Since $R \subseteq I_j$, we have $I_j^{-1} \subseteq R$, and hence $J_{ij} = I_i I_j^{-1} \subseteq I_i$. Thus, for every nonzero $u \in J_{ij}$, we have $(u) \subseteq I_i$, and so

$$|N_{K/\mathbb{Q}}(u)| = N((u)) \geq N(I_i) \geq 2^{-d^2} \Delta_K^{-1/2}.$$

By the arithmetic-geometric mean inequality applied to the canonical embedding,

$$\|u\| \geq \sqrt{d} |N_{K/\mathbb{Q}}(u)|^{1/d} \geq \sqrt{d} 2^{-d} \Delta_K^{-1/(2d)} := \eta_K.$$

Then every nonzero vector $u_{ij,t} \in J_{ij}$ satisfies $\|u_{ij,t}\| \geq \eta_K$. Hence, for every s ,

$$\prod_{t=1}^d \|u_{ij,t}\| = \|u_{ij,s}\| \prod_{t \neq s} \|u_{ij,t}\| \geq \|u_{ij,s}\| \eta_K^{d-1}.$$

Taking the maximum over s gives

$$\max_t \|u_{ij,t}\| \leq \eta_K^{-(d-1)} \prod_{t=1}^d \|u_{ij,t}\|.$$

Combining this with the preceding product bound gives

$$\max_t \|u_{ij,t}\| \leq \eta_K^{-(d-1)} 2^{\frac{d(d-1)}{2}} \Delta_K^{1/2} N(J_{ij}).$$

Using $N(J_{ij}) = \frac{N(I_i)}{N(I_j)} \leq 2^{d^2} \Delta_K^{1/2}$, we also obtain

$$\max_t \|u_{ij,t}\| \leq \eta_K^{-(d-1)} 2^{2d^2} \Delta_K.$$

Therefore

$$\|e_{ij}\| \leq d \eta_K^{-(d-1)} 2^{2d^2} \Delta_K =: C_K.$$

In particular,

$$\log C_K = O(d^2 + d \log d + \log \Delta_K),$$

and hence $\log C_K$ is polynomially bounded in d and $\log \Delta_K$.

Fix j , and during the inner loop define

$$m_j := \max_{i' < j} \left\| \frac{r_{i'j}}{r_{i'i'}} \right\|.$$

Consider one update at index $i < j$. Since

$$x_i := x_{ij} = \frac{r_{ij}}{r_{ii}} - e_{ij},$$

we get

$$\|x_i\| \leq \left\| \frac{r_{ij}}{r_{ii}} \right\| + \|e_{ij}\| \leq m_j^{\text{old}} + C_K.$$

For any $i' < i$, the update $b_j \leftarrow b_j - x_i b_i$ transforms $r_{i'j}$ into

$$r'_{i'j} = r_{i'j} - x_i r_{i'i}.$$

Hence

$$\left\| \frac{r'_{i'j}}{r_{i'i'}} \right\| \leq \left\| \frac{r_{i'j}}{r_{i'i'}} \right\| + \|x_i\| \left\| \frac{r_{i'i}}{r_{i'i'}} \right\|.$$

But the i -th column has already been size reduced, so

$$\left\| \frac{r_{i'i}}{r_{i'i'}} \right\| \leq C_K.$$

Therefore

$$m_j^{\text{new}} \leq m_j^{\text{old}} + \|x_i\| C_K \leq (1 + C_K) m_j^{\text{old}} + C_K^2.$$

Iterating over at most n values of i , we obtain

$$m_j \leq (1 + C_K)^n (m_j^{\text{init}} + C_K).$$

Since $\log C_K$ is polynomially bounded in $\log \Delta_K$, it follows that $\log m_j$ is polynomially bounded as soon as $\log m_j^{\text{init}}$ is. Since the initial Gram–Schmidt coefficients are determined by the input pseudo-basis, their logarithmic Archimedean sizes are bounded by a polynomial in the input bit-length L , n , and d . Hence

$$\log m_j^{\text{init}} \leq \text{poly}(L, n, d).$$

Combining this with the preceding inequality gives

$$\log m_j \leq \text{poly}(L, n, d, \log \Delta_K).$$

Finally, since

$$\|b_j\|^2 = \sum_{i \leq j} \|r_{ij}\|^2,$$

we have

$$\|b_j\| \leq \sqrt{n} \max_{i \leq j} \|r_{ij}\|.$$

For $i < j$, by the definition of m_j ,

$$\left\| \frac{r_{ij}}{r_{ii}} \right\| \leq m_j,$$

and therefore

$$\|r_{ij}\| = \left\| \frac{r_{ij}}{r_{ii}} r_{ii} \right\| \leq \left\| \frac{r_{ij}}{r_{ii}} \right\| \|r_{ii}\| \leq m_j \max_{i \leq j} \|r_{ii}\|.$$

For $i = j$, trivially

$$\|r_{jj}\| \leq \max_{i \leq j} \|r_{ii}\|.$$

Hence

$$\max_{i \leq j} \|r_{ij}\| \leq \max_{i \leq j} \|r_{ii}\| \max(1, m_j),$$

so that

$$\|b_j\| \leq \sqrt{n} \max_{i \leq j} \|r_{ii}\| \max(1, m_j).$$

Because the pseudo-basis is scaled, each diagonal coefficient satisfies

$$\|r_{ii}\| \leq 2^d \Delta_K^{1/(2d)} N(r_{ii} I_i)^{1/d},$$

and the quantities $N(r_{ii} I_i)$ are unchanged throughout the algorithm. Indeed, the ideal-compatible size reduction step modifies only the off-diagonal coefficients and leaves both r_{ii} and I_i unchanged. On the other hand, the scaling step replaces I_i by $I_i \langle x_i \rangle^{-1}$ and b_i by $x_i b_i$, so that r_{ii} is replaced by $x_i r_{ii}$. Hence $(x_i r_{ii})(I_i \langle x_i \rangle^{-1}) = r_{ii} I_i$, and therefore $N(r_{ii} I_i)$ is invariant. Hence $\log \max_{i \leq j} \|r_{ii}\|$ is polynomially bounded by the input bit-length and $\log \Delta_K$. Combining this with the bound on m_j shows that $\log \|b_j\|$ always remains below a polynomial in the input bit-length, n , and $\log \Delta_K$, as claimed. $\square$

Remark 4. The bit-size need not decrease monotonically after the scaling step or the size reduction step. What is required is that the bit-sizes remain bounded by a polynomial in the input bit-size, n , d , and $\log \Delta_K$ throughout the algorithm.

The coefficient ideals I_i affect the vectors b_j only indirectly, through the reduction coefficients. In the corrected size reduction step, the coefficient x_{ij} is chosen in the fractional ideal $I_i I_j^{-1}$. Hence the effect of the update

$$b_j \leftarrow b_j - x_{ij} b_i$$

is controlled by the geometry of the lattice $I_i I_j^{-1}$. In particular, once the pseudo-basis is scaled, the norms $N(I_i)$ are uniformly bounded above and below, which yields a uniform control on $N(I_i I_j^{-1})$, and therefore on the growth of the vectors b_j .

C Detailed comparison with the state of the art

In this section, we provide a detailed comparison between adelic LLL and the module LLL algorithm of [31], the state-of-the-art fully structured reduction algorithm to date. Specifically, we clarify (i) where the core differences lie (ii) what oracles are used (iii) what assumptions are needed (iv) what it takes to dequantize (v) how their output qualities compare. We focus on their rank 2 reductions — Algorithm 8 above and [31, Algorithm 4.3] — since the differences between their rank n versions are relatively immaterial.

Unravelling the notations, the two algorithms are in fact similar in a few important ways. Both algorithms make progress by repeating size reduction (which [31] calls a “Euclidean division”) and swap, like the classical LLL algorithm. Moreover, their swaps — Algorithm 4 and lines 4-5 of [31, Algorithm 4.3] — are practically identical. (Still, it may be worth noting that the adelic language transparently reveals that it is indeed a swap, in that it literally swaps the rows of the input matrix $g \in \mathrm{GL}(2, \mathbb{A})$.)

One minor difference — that may matter for implementations however — is that [31] uses the so-called Siegel condition (line 1 of [31, Algorithm 4.3]) for the Lovász test, whereas we use the adelic generalization of the original Lovász condition (Definition 1(ii)), which is in principle sharper than the Siegel condition. It is easy to write a Siegel version of our algorithm if desired; the corresponding proofs of Propositions 1 and 2 differ only in a very minor way. It seems less clear, however, how to adjust the algorithms of [31] to accommodate the Lovász condition.

More importantly, the core difference between adelic LLL and [31, Algorithm 4.3] lies in the specific Diophantine problems associated with their size-reduction routine. This is what gives rise to all their differences on the required oracles and assumptions, and so on. We elaborate on this point below.

C.1 Differences in the Diophantine conditions

The size reduction of adelic LLL (Algorithm 2) aims to solve the following adelic Diophantine approximation problem: given the parameters $(\tau_\sigma)_{\sigma: F \hookrightarrow \mathbb{C}}, 0 < \mu < 1$ and

a target $m \in \mathbb{A}_F$, find $r \in F$ such that

$$\tau_\sigma |m_\sigma - r_\sigma| \leq \mu \quad \forall \sigma : F \hookrightarrow \mathbb{C}, \text{ and } H_{\mathbf{f}}(m - r, 1) \leq \prod_{\sigma} \tau_\sigma. \quad (11)$$

In the reduction of a pseudobasis, one always has $m_{\mathbf{f}} = 0$, for which it suffices to solve the following simpler version: find $p, q \in \mathcal{O}_F$ such that

$$|q_\sigma m_\sigma - p_\sigma| \leq \mu \text{ and } |q_\sigma| \leq \tau_\sigma \quad (12)$$

(cf. (7) above) for all embeddings σ . We want to set τ_σ so as to minimize $\prod_{\sigma} \tau_\sigma$ while ensuring (11) is always solvable (within a given time budget). In our paper, we set all τ_σ to (9).

In comparison, the counterpart in [31] — see Algorithm 4.1, and also Corollary 4.8 therein — is the following problem. For parameters $\varepsilon, C, c > 0$ and inputs $a \in (F \otimes \mathbb{R})^*$, $b \in F \otimes \mathbb{R}$, and an ideal $\mathfrak{a}$ satisfying $c^{-d} \leq N(\mathfrak{a}) \leq c^d$, find $(u, v) \in \mathcal{O}_F \times \mathfrak{a}$ such that

$$\|ua + bv\|_\infty \leq \varepsilon \|a\|_\infty, \text{ and } \|v\|_\infty \leq C. \quad (13)$$

Here $\|x\|_\infty = \max_{\sigma} |\sigma(x)|$ for $x \in F$. This can also be seen as a kind of a Diophantine approximation problem. [31] chooses $\varepsilon = 1/(4c)$ ([31, Algorithm 4.3]) and $C = c^{2^{0.55\delta B/d}}$ ([31, Theorem 4.6]), but the constant c , which originates from [31, Lemma 2.5], is only stated to be $2^{\tilde{O}(\log \Delta_F)/d}$. An explicit formula, such as the one for our τ_σ 's, seems to be unavailable, which impacts the implementability of the algorithm of [31].

All the major differences between adelic LLL and [31, Algorithm 4.3] originate from the fact that (12) is a much easier problem to solve than (13), for which even the correct parameter value is unclear. All the CVP oracles, heuristic assumptions, and quantization issues of [31] appear precisely to claim that (13) is solvable. We examine each of these components in the subsequent sections.

C.2 Subroutines needed

Adelic LLL can run on an oracle as weak as dimension $d + 1$ classical LLL — see Algorithm 2 and its accompanying explanations. For a better output quality, one could choose a stronger HSVP algorithm in dimension $d + \deg E$ for a subfield $E \subseteq F$.

On the other hand, the rank-2 reduction of [31, Alg 4.3] requires two subroutines:

- [31, Alg. 4.2] to scale a pseudo-basis: this is a heuristic quantum algorithm which, given as input a fractional ideal I of R and any $\alpha \in F_{\mathbb{R}}^\times$, outputs $x \in \alpha I \setminus \{0\}$ such that

$$\|x\|_\infty \leq c \cdot |N(\alpha)|^{1/d} \cdot N(I)^{1/d},$$

where

$$c = 2^{\tilde{O}(\log \Delta_F)/d}.$$

If $\alpha \in F$, the algorithm performs a number of quantum operations that is polynomial in $\log \Delta_F$ and the input bit-length, and makes a single call to a CVP oracle in a lattice $L_{F,1}$ of dimension $\tilde{O}(\log \Delta_F)$ and depending only on F .

- [31, Alg. 4.1] which is some kind of Euclidean division over $\mathcal{O}_F$. The algorithm is heuristic and calls Alg. 4.2 three times, but also makes a single call to a CVP oracle in $\mathcal{O}_F$, and another single call to a CVP oracle in a lattice $L_{F,2}$ of dimension $O((d \log \rho(\mathcal{O}_F))^{2+\eta})$ for any $\eta > 0$

Even with preprocessing, CVP is known to be NP-hard [26]. The lattices $L_{F,1}$ and $L_{F,2}$ only depend on the field F , but the target is different for each instantiation. Hence the rank 2 algorithm of [31] requires subroutines to solve instances of NP-hard problems.

In addition, the dimensions of these lattices are prohibitively large. $L_{F,1}$ has dimension $\tilde{O}(\log |\Delta_F|)$. For cyclotomic fields this is $\tilde{O}(d)$. According to [31, Corollary 4.8], $L_{F,2}$ has dimension $O_\eta(d^{2+\eta})$ for $\eta > 0$. To dequantize the algorithm of [31], $L_{F,1}$ need to be replaced by $L'_{F,1}$, whose dimension is almost as large as that of $L_{F,2}$.

C.3 Heuristic assumptions

Adelic LLL requires no heuristics.

The two heuristic pillars of [31] are Heuristic 1 and Lemma 2.5 (heuristic) of their paper. Lemma 2.5 in turn assumes the generalized Riemann hypothesis and Heuristic 4 of [36]. These are independent heuristics, and every other heuristic statement of [31] follows from either of these two. Heuristic 1 is a statement to the effect that the CVP target $\mathbf{t}$ in [31, Algorithm 4.1] is close enough to $L_{K,2}$ for a solution to (13) with respect to the claimed parameters to exist. Lemma 2.5 is a statement that, with one call to the CVP oracle of $L_{K,1}$, it can find a short vector of a weighted ideal in polynomial time, which is used as a supplementary device in their Algorithm 4.1.

In pages 4-5 of [31], an approach to make its rank 2 algorithm heuristic-free is suggested. Briefly speaking, they reduce their problem to 3SAT which they again reduce to CVP with preprocessing for a lattice construction that would play the role of $\mathcal{O}_F$, $L_{F,1}$ and $L_{F,2}$ combined. However, this construction has an even higher dimension than $\mathcal{O}_F \oplus L_{F,1} \oplus L_{F,2}$, and by a far margin.

C.4 Dequantization

Adelic LLL is classical as presented in this paper.

Lemma 2.5 (heuristic) of [31], on the other hand, entails a quantum algorithm. According to [31], it is dequantizable with additional heuristics (Heuristics 2 and 3 of [36]) and subexponential running time disregarding the complexity of the CVP oracle; see the remarks below Theorem 1.2 and Lemma 2.5 of [31].

One does not need to take this path for the dequantization of [31], however. For a free rank 2 module with a known basis, [31, Algorithm 4.3] can be made to run on classical computers, by replacing [31, Lemma 2.5] with [31, Lemma 5.2]. On the other hand, [10] gives a probabilistic polynomial-time reduction of the problem of reducing an arbitrary rank 2 module to that of reducing a free rank 2 module. Combining, one obtains a dequantization of [31, Algorithm 4.3]. All the dependencies on the oracles and the heuristics remain; in fact, it becomes worse in that $L_{K,1}$ needs to be replaced by the much larger $L'_{K,1}$.

C.5 Performance comparison

Unraveling the definitions, our Q from Proposition 2 is to be compared against $\alpha_F^{1/2}$ of [31]. From the proof of [31, Corollary 4.2], one has $\alpha_F \approx 2^d \gamma_{\mathcal{N}}^{2d} |\Delta_F|$, and from [31, Theorem 4.1], $\gamma_{\mathcal{N}} = 2^{\tilde{O}(d \log \rho)/d}$, where ρ here is the covering radius of the Minkowski embedding of $\mathcal{O}_F$. In summary,

$$\alpha_F^{1/2} \approx 2^{\tilde{O}(d \log \rho)} \sqrt{|\Delta_F|}.$$

This is much better than Q , which is $2^{O(d^2)}$, very roughly speaking (a more precise formula in case $F = \mathbb{Q}(\zeta_{2^k+1})$ is discussed in Section 4.3). It may be helpful to compare in terms of the root Hermite factor also. Given a rank 1 submodule of height $QH(\det g)^{1/2}$, one can apply any reduction algorithm to it to obtain a vector of length $2^{O(d)} H(\det g)^{1/(2d)}$; replacing Q by $\alpha_F^{1/2}$, the number becomes $2^{O((\log d)^k)} H(\det g)^{1/(2d)}$ for some $k > 0$ implicit in the $\tilde{O}$ notation, comparable to HSVP with a quasi-polynomial approximation factor.

One is reminded, however, that this advantage comes at a cost of polynomially many exact CVP instances on a lattice of dimension $O(d^{2+\eta})$, as discussed above.

D Reductions to Rank-2

For comparison, we recall the reductions to rank-2 proved in [31,34]:

- [34, Theorem 5.10] states three reductions: For any number field K , order $R \subseteq \mathcal{O}_K$ of degree n with an inner product $\langle \cdot, \cdot \rangle_\rho$ over $K_{\mathbb{R}}$, rank $k \geq 2$, approximation factor $\gamma = \gamma(R, k) \geq 1$, semicanonical inner product $\langle \cdot, \cdot \rangle_\rho$, and constant $\varepsilon > 0$, there is an efficient reduction from (γ, k) -ModuleSVP to $(\gamma', 2)$ -ModuleSVP, where

$$\gamma := (1 + \varepsilon) \cdot \left(\frac{\gamma' \delta_n}{\alpha_R} \right)^2 \cdot \left(\frac{\gamma' \delta_n \mu_{R,2}}{\alpha_R} \right)^{2(k-2)}.$$

There is also an efficient reduction from (γ_H, k) -ModuleHSVP to $(\gamma', 2)$ -ModuleSVP, where

$$\gamma_H := \gamma' \delta_n \cdot \left((1 + \varepsilon) \gamma' \frac{\delta_n \mu_{R,2}}{\alpha_R} \right)^{k-1}.$$

For any number field K , order $R \subseteq \mathcal{O}_K$, rank $k \geq 2$, approximation factor $\gamma' = \gamma'(R, k) \geq 1$, semicanonical inner product $\langle \cdot, \cdot \rangle_\rho$, and constant $\varepsilon > 0$, there exists an efficient reduction from (γ, k) -DIP to $(\gamma', 2)$ -DIP, where

$$\gamma := (1 + \varepsilon) \gamma' \cdot ((1 + \varepsilon) \gamma' \mu_{R,2})^{2(k-2)}.$$

where:

Hermite's constant

$$\delta_n := \sup_L \frac{\lambda_1(L)}{\det(L)^{1/n}}.$$

Other constants

$$\alpha_R := \inf_I \frac{\lambda_1(I)}{\det(I)^{1/n}},$$

where the infimum is over all rank-one modules $I \subset K_{\mathbb{R}}^l$. For the canonical embedding, we have

$$\alpha_R = \frac{\sqrt{n}}{\det(R)^{1/n}}.$$

$$\mu_{R,k} := \sup_M \frac{\tau_1(M)}{\det(M)^{1/(kn)}},$$

where the supremum is over all rank- k module lattices $M \subset K^{k'}$ for any integer $k' \geq k$, and for any module lattice M

$$\tau_1(M) := \min_{I \subset M} \det(I)^{1/n},$$

where the minimum is over the rank-one submodule lattices. Then

$$1 \leq \mu_{R,k} \leq \frac{\delta_{kn}}{\alpha_R}. \quad (4)$$

(γ, k) -ModuleSVP For an approximation factor $\gamma = \gamma(R, k) \geq 1$, the input is (a generating set for) a module lattice $M \subset K_{\mathbb{R}}^l$ with rank k , the goal is to output a module element $x \in M$ such that

$$0 < \|x\|_{\rho} \leq \gamma \lambda_1(M).$$

(γ, k) -ModuleHSVP For an approximation factor $\gamma = \gamma(R, k) \geq 1$, the input is (a generating set for) a module lattice $M \subset K_{\mathbb{R}}^l$ with rank k , the goal is to output a module element $x \in M$ such that

$$0 < \|x\|_{\rho} \leq \gamma \det(M)^{1/(kn)}.$$

(γ, k) -DIP For an approximation factor $\gamma = \gamma(R, k) \geq 1$, the input is (a generating set for) a module lattice $M \subset K_{\mathbb{R}}^l$ with rank k , the (γ, k) -Dense Ideal Problem, or (γ, k) -DIP, is asks to find a rank-one submodule $M' \subset M$ such that

$$\det(M')^{1/n} \leq \gamma \tau_1(M).$$

- [31, Th 3.9] states: Let $\gamma \geq 1$ and assume that an LLL-reduced $\mathbb{Z}$ -basis of R is known. Then there exists a reduction from solving $\text{SVP}_{\gamma'}$ in rank- n modules (with respect to $\|\cdot\|$) in K^n to solving SVP_{γ} in rank-2 modules in K^2 , where

$$\gamma' = \left(2\gamma \Delta_K^{1/d}\right)^{2n-1}.$$

This reduction runs in time polynomial in $\log \Delta_K$ and the bit-length of the input pseudo-basis.

Hence, all previous rank-2 reductions [31,34] transformed a computational problem (either HSVP or approx-SVP) in a module lattice of arbitrary rank into approximating the shortest vector in a rank-2 module lattice. By contrast, we reduce such problems to finding a short vector in a rank-1 module lattice, with respect to its determinant: due to the structure of rank-1 module lattices, such a vector also approximates a shortest vector.